ONE MANUSCRIPT, END TO END

# A Book, Start to Finish

*One Manuscript from Blank Project to Published Page*

Vladimir Ulogov

2026 · examples assume Inkhaven 3.0.0 or newer

# Contents

# Following One Book

The companion volume, **The Inkhaven Manual**, is a reference: it takes each feature in turn and tells you what it is and how to run it. This book does the opposite. It picks **one manuscript** and follows it — from the empty project on a Monday morning to a typeset PDF and a small web edition — reaching for each feature exactly when a real author would, and no sooner.

If you learn best by watching something get built, start here. You can read this book beside the manual (it points to the relevant chapter whenever it uses a feature in passing) or entirely on its own.

## The book we are writing

Our example is a short fantasy mystery — long enough to exercise a world, a cast, and a secret; short enough to finish inside one volume.

> TERM **The Ninth Lantern**
>
> Nine lanterns ring the harbour town of Saltmarch, and for three hundred years no keeper has let one go dark. On the morning the story opens, the ninth lantern is cold — and the keeper who tended it is gone. A short mystery: a small town, a handful of suspects, a magic that runs on light, and a truth that someone has worked very hard to keep.

It is deliberately a work of fiction with **secrets and a cast**, because that is what exercises the most of Inkhaven — a world to keep consistent, characters who must each sound like themselves, and a reveal that no one may act on too early. If you write non-fiction, the **shape** of the journey is identical; where a step is fiction-specific (the world, who-knows-what, the voices) the text notes the non-fiction parallel.

## The shape of the journey

*From blank project to published book*

```
I    The Foundation ..... init the project · sketch the world & cast
II   Drafting ........... write the opening · keep the facts straight
III  The Middle ......... plant the secret (KEN) · voices & threads
IV   Revision ........... the read-through · the editorial pass ·
                          did it get better?
V    Publishing ......... assemble the PDF · EPUB & the web edition
```

Each part is a stage every book passes through. You will not use every Inkhaven feature to write **your** book — few books need all of them — but by the end of this one you will have seen where each fits in the arc of a manuscript, and be able to reach for the right one at the right time.

**WHAT YOU LEARNED**

- This book follows **one manuscript**, "The Ninth Lantern", from `inkhaven init` to a published PDF — learning by watching a book get written.
- It is a companion to **The Inkhaven Manual**: the manual is the reference, this is the journey. Read either alone or side by side.
- The example is fiction with a world and a secret because that exercises the most of Inkhaven; the **arc** is the same for non-fiction, and the text notes the parallels.

PART I

# The Foundation

CHAPTER 1

# 1

# Starting the Project

Every book begins the same way: with nothing. A title you are half-sure of, a first scene you can almost see, and an empty place to put them. This chapter is about making that empty place — turning "I want to write **The Ninth Lantern**" into a project on disk you can open tomorrow morning and the morning after that.

We will run exactly one command, open what it made, and build the first three rungs of the book — a book, a chapter, a paragraph — ending with a single sentence saved to disk as plain text. It is a small amount of work. The point of doing it slowly, once, is that everything else in this book happens **inside** the thing we make here.

> **THIS IS THE JOURNEY, NOT THE REFERENCE**
>
> Chapter 2 of **The Inkhaven Manual** names every file `init` writes and every one of the twenty seeded system books. This chapter does not repeat that anatomy — it **starts a specific book** and points to the manual when you want the full map. Read them side by side if you like; you do not need to.

## One command

Open a terminal, go to wherever you keep your writing, and run `inkhaven init` with the name of the project. That name becomes a directory — one project per book is the convention — so pick the slug you want to live with. Ours is `the-ninth-lantern`.

*Starting The Ninth Lantern*

```
$ cd ~/Books
$ inkhaven init the-ninth-lantern
Initialized inkhaven project at
  /home/you/Books/the-ninth-lantern
  config:    …/the-ninth-lantern/inkhaven.hjson
  prompts:   …/the-ninth-lantern/prompts.hjson
  store db:  …/the-ninth-lantern/metadata.db
  vecstore:  …/the-ninth-lantern/vectors
  books:     …/the-ninth-lantern/books
```

That is the whole of it. The directory did not exist a moment ago; `init` made it and laid down a small, legible project inside. Two of those files are yours to edit by hand — `inkhaven.hjson` (theme, language, AI provider) and `prompts.hjson` (your prompt library) — and the rest are the local store: `metadata.db` remembers the **shape** of the book, `vectors/` powers semantic search, and `books/` is where your actual prose will live, one folder per book, as ordinary text files you could read with any editor on earth.

> **THE FIRST INIT IS SLOWER**
>
> The very first time you ever run `init` on a machine, Inkhaven downloads a small multilingual embedding model (about 120 MB) into a shared per-user cache — not into the project. It is what makes search and the reading intelligences work, entirely on your own computer, offline. Every project after the first reuses it and starts instantly.

We took the default, which is an **empty** project — the system books and nothing else. There is a `--template novel` that would have scaffolded a three-act skeleton and a Characters stub for us, and it is a fine shortcut. We are declining it on purpose: building the first few rungs by hand, once, teaches the tree better than any template can, and **The Ninth Lantern** is short enough that we lose nothing by starting from bare stone.

## Opening the empty project

A project on disk is inert until you open it. Running `inkhaven` with no arguments opens whatever project sits in the current directory, so step in and launch it.

*Opening the editor*

```
$ cd the-ninth-lantern
$ inkhaven
```

The full-screen editor comes up. Chapter 3 walks its panes properly; for now notice only that the left-hand **Tree** has focus, and that the project you believed was empty is not. A stack of books is already there — the **system books**, seeded into every project, protected so you cannot delete them by accident, waiting to hold things you have not written yet: your cast, your places, your sources, your facts. Anything you make lands **above** them.

● ● ● *A fresh project — the system books, waiting*

```
  TREE ──────────────────────────────────
▸ Notes            [book · notes]
▸ Research         [book · research]
▸ Sources          [book · sources]
▸ Facts            [book · facts]
▸ Places           [book · places]
▸ Characters       [book · characters]
▸ Threads          [book · threads]
  …thirteen more system books…
```

This is worth a moment's respect before we write anything. Saltmarch will need a **gazetteer** — the Long Mole, the nine pillars, the quays — and it already has a home in **Places**. Mira and Aldous and the harbourmaster will need a **cast list**, and **Characters** is standing ready. The secret at the centre of the book — that Aldous put the lantern out himself — will one day be checked against **Facts** so no one acts on it too early. None of that is our concern this morning. The point is only that when each of those needs arrives, the shelf for it already exists, tagged the same in this project as in every project you will ever open.

### FICTION

The world books — **Places**, **Characters**, **Facts**, **Threads** — are where a novel keeps its ground truth. You will feed them as you invent, and in return the editor highlights your names and the continuity readers get something to watch. We meet them properly in Part I's next chapters.

### NON-FICTION

The same shelves serve an argument. **Facts** holds a non-fiction book's verified claims instead of a world's invariants; **Sources** becomes your bibliography; **Research** holds the material you gathered. The book you are writing changes what goes on the shelves, never that they are there.

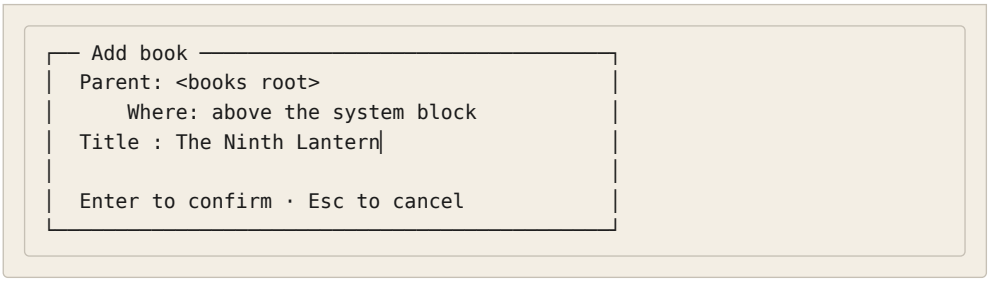
## The first book

The Tree is driven by single letters — one per level of the hierarchy. To lay down the spine of a manuscript you need only three of them, plus Enter to open a leaf for editing.

| | |
|---|---|
| **B** | add a book at the root, above the system books |
| **C** | append a chapter under the selected book |
| **+** | append a paragraph under the selected branch |
| **Enter** | open the selected paragraph in the Editor |
| **F2** | rename the selected node |

Press `B`. A small dialog asks for a title. This is the book itself — the top of the tree, the thing the whole project is named for — so give it the real title.

*Naming the book*




Press Enter, and **The Ninth Lantern** appears at the very top of the tree, sitting above `Notes` where every book of yours will sit. It is empty — a titled branch with nothing beneath it — but it is the book, and the cursor is on it.

## The first chapter, the first scene

With the cursor on the book, press `C` to hang a chapter beneath it. The story opens on the morning the ninth lantern is found cold, so that is the chapter's name — "The Cold Lantern." Then, with the cursor on the chapter, press `+` to add a paragraph

under it. You may leave the paragraph's title blank; Inkhaven will name it from your first sentence when you save, which for an opening scene is exactly what you want.

*The spine of the book, three rungs deep*

```
  TREE ──────────────────────────────
▾ The Ninth Lantern
  ▾ The Cold Lantern
      ¶ Untitled paragraph
▸ Notes            [book · notes]
▸ Research         [book · research]
▸ Sources          [book · sources]
  …the other system books…
```

That little tree is the whole model in miniature. The book is a folder. The chapter is a numbered folder inside it. The paragraph is a numbered `.typ` file inside **that** — the leaf, the actual unit of prose you edit, versioned and searched on its own. A "paragraph" can hold a whole scene despite the name; think of it as the smallest titled section Inkhaven keeps track of. All of it is plain Typst text under `books/`, and the database beside it merely points at the files.

> **WHERE THIS LIVES ON DISK**
>
> After the next save, the tree above is mirrored one-to-one under `books/`: `books/the-ninth-lantern/01-the-cold-lantern/01-<slug>.typ`. Folders are branches, files are leaves, and the zero-padded numbers keep a plain directory listing in reading order. Chapter 2 of the manual dissects that mirror in full.

## The first words

Put the cursor on the paragraph row and press Enter. Focus jumps to the Editor in the middle pane, which shows the paragraph's starting template — a single Typst heading line and an empty body waiting below it.

*A new paragraph, before a word is written*

```
  EDITOR ──────────────────────────────
= Untitled paragraph

|
```

Press `End` to reach the end of the heading, Enter twice to leave it behind, and write the first line of the book. It has been waiting since we picked the title.

*The opening line of The Ninth Lantern*

```
  EDITOR ──────────────────────── ● edited
= Untitled paragraph

On the morning the ninth lantern went cold, Mira
Fenn was the first in Saltmarch to see it — a dark
pillar at the end of the Long Mole where, for three
hundred years, there had always been a flame.|
```

The moment you touched the buffer the border around the Editor turned **yellow** — Inkhaven's plain signal that there is unsaved work. Press `Ctrl+S`. Three things happen at once and none of them ask you anything: the `.typ` file is written under `books/`, its metadata is updated, and a fresh embedding is computed so the sentence is findable by meaning as well as by word. The border returns to **green**.

Because we left the paragraph's title empty, one more thing happened — Inkhaven took the title from the opening sentence, renamed the file on disk to match, and the Tree row now reads it back to us.

*Saved — the leaf names itself*

```
  TREE ────────────────────────────────
▾ The Ninth Lantern
  ▾ The Cold Lantern
      ¶ On the morning the ninth lantern went…
▸ Notes            [book · notes]
```

That is a complete round trip: a book, a chapter, a scene, saved to disk as plain Typst, indexed for search, and named after its own first line. It is also the whole loop you will run thousands of times — open a leaf, write, `Ctrl+S`, watch the border go green — for the rest of the manuscript. Everything else this book teaches is something you reach for **around** that loop.

To stop for the day, press `Ctrl+Q`. Inkhaven saves anything still dirty and records the session, so tomorrow's `inkhaven` reopens this project with the cursor back on this very paragraph. The blank project is no longer blank.

## A word about shape, before you fill it

You now have one book, one chapter, one scene. Resist, for a moment, the urge to pour in the next ten. The tree you build is the outline you will write against, and it is worth a minute's thought about how to grow it — a difference that falls out along the fiction / non-fiction line and recurs through the whole book.

**FICTION**

Grow the tree as the **spine of the story**. One chapter per act or major movement of **The Ninth Lantern** — the cold lantern, the wrong suspicion, the walk onto the Mole, the reveal, the choice — with paragraphs as scenes beneath them. Drop names into **Characters** and **Places** as you invent them so the prose lights them up. The `novel` template lays exactly this down if you would rather not start from stone.

**NON-FICTION**

Grow the tree as the **table of contents**. A chapter per part, subchapters for the sections beneath — the tree **is** the outline, and it becomes the document's structure verbatim. Put gathered material into **Research** and citations into **Sources** from day one, so every claim has a home and a bibliography assembles itself. The `nonfiction` and `technical` templates scaffold this shape.

Neither shape is a promise. The tree reorders freely — rows move up and down, paragraphs move between chapters, whole branches relocate — and every move renumbers the files on disk for you, so you never think about the numbers at all. Build the shape you can see this morning. Reshape it the moment the book tells you to, which, if **The Ninth Lantern** is anything like every book before it, will be soon.

**WHAT YOU LEARNED**

- `inkhaven init the-ninth-lantern` makes a project directory and lays down a small, legible project: `inkhaven.hjson` and `prompts.hjson` are yours to edit; `metadata.db` / `vectors/` are the store; `books/` holds your prose as plain Typst. The first init on a machine downloads a shared embedding model to a per-user cache, not into the project.
- `inkhaven` with no arguments opens the project in the current directory. A fresh project is not empty — it is pre-seeded with protected **system books** (Notes, Research, Sources, Facts, Places, Characters, and more) that hold your world's ground truth, and your own books sit above them.
- The Tree is driven by single letters: `B` makes a book, `C` a chapter, `+` a paragraph, Enter opens a leaf, `Ctrl+S` saves, `Ctrl+Q` quits. We built **The Ninth Lantern** → **The Cold Lantern** → the opening scene, mirrored to disk as folders and a numbered `.typ` file.
- Saving computes a fresh embedding, writes the `.typ` file, and — for an untitled leaf — names it from the first sentence and reopens exactly there next launch. Grow the tree as the **spine of a story** or as a **table of contents**; either way it reorders freely and renumbers the files for you. The manual's Chapter 2 has the full on-disk anatomy.

CHAPTER 2

# 2

# The World and the Cast

We have an empty project with a book in it and nothing written. It is tempting to start with a sentence. Resist it for one more chapter. Before **The Ninth Lantern** has a first line, it needs two things a mystery cannot do without: a **place** for the ninth lantern to go dark in, and **people** to notice. We are not going to build them exhaustively — the story is a scaffold, and the bible's first instruction is to keep it light — but we are going to write down just enough that Inkhaven can start watching our world for us. Ten minutes of typing now buys the overlay, the grounded assistant, and — chapters from here — the continuity reader, the voice reader, and the who-knows-what reader, all at once.

This is the payoff of doing it first. A name you record is a name every reading intelligence in the tool can see; a name you never record is a name none of them can. So we spend this chapter filling two small books that were seeded for us at `init`, and then we watch the editor light up.

## Two books were already waiting

Look back at the Tree pane from the last chapter. Below our own book and above Help sits a small block of **system books** — Notes, Research, Prompts, and two that this chapter is about: **Places** and **Characters**. Inkhaven created them at `init`. They are ordinary books — chapters, paragraphs, Typst prose — but Inkhaven reserves them, keeps them out of the finished manuscript, and wires behaviour to what you type into them.

● ● ● *The Tree — our book above, the seeded system block below*

```
The Ninth Lantern        (our book)
──────────────────────────────────

Notes
Research
Prompts
Places          ← cyan overlay · Ctrl+B P
Characters      ← yellow overlay · Ctrl+B C
Help
```

The two are structural twins. Everything below about a Place is true of a Character with three swaps: the book, the overlay colour (cyan for Places, yellow for Characters), and the RAG chord (`Ctrl+B P` versus `Ctrl+B C`). The data model is the whole of one sentence: **an entry is a paragraph, and the paragraph's title is the entity's name.** There is no "register" step — saving a paragraph is registering it.

> **TERM** **System book**
>
> A book Inkhaven seeds and reserves — `places`, `characters`, `facts`, and a few more. You freely add, edit, and delete the paragraphs *inside* it, but you cannot delete the book itself. Its contents drive a feature instead of appearing in your PDF, and the prose tools leave it out of the corpus.

## Sketching Saltmarch

Move the tree cursor to **Places**, press `→` to expand it, and press `+` to add a paragraph. The Add modal asks for a title; that title **is** the place name. Press Enter, and the new paragraph opens in the editor ready for whatever you want to remember — description, history, why it matters. It is ordinary Typst: headings with `=`, `*bold*`, lists, all of it.

Saltmarch is small, so its map is small. We give it the town, the harbour it rings, the mole that reaches into the water, and the two lanterns that matter — the nine as a set, and the ninth that stands alone at the end of the mole. Five flat paragraphs, no grouping; with a cast of one town we do not need chapters yet. (When a Places book grows to dozens of entries, you would add chapters — `Cities`, `Regions`, `Buildings` — and nest under them. Not here.)

● ● ● *The Places book for The Ninth Lantern*

```
Places
  Saltmarch               (paragraph)
  The harbour             (paragraph)
  The Long Mole           (paragraph)
  The nine lanterns       (paragraph)
  The ninth lantern       (paragraph)
```

Each body stays a few lines — the bible says keep it light, and a short entry is enough for the overlay and plenty for the assistant to ground on. Saltmarch's looks like this:

● ● ● *Editor · Places / Saltmarch — the entry body*

```
= Saltmarch

A poor fishing town on a cold northern coast. Stone
quays, a working harbour, sea-fret off the water most
mornings. Nine lanterns ring the harbour; the creed
is that no lantern goes dark.
```

**ONE ENTRY PER PARAGRAPH**

Keep entries atomic. The overlay, the grounded lookup, and later readers all key off the paragraph **title**, so a paragraph that describes two places is only ever found under one name. If you catch yourself writing "the harbour and the mole" as one entry, split it — that is exactly why **The Long Mole** is its own paragraph here.

## Adding the cast

Characters works identically. Move to the **Characters** row, `→` to expand, `+` to add, and title each paragraph with the character's **canonical name**. Our cast is five, so again we stay flat — most authors group a larger cast by role (Protagonists, Supporting) or by the family they belong to, but five names read fine as a list.

*The Characters book — the cast of The Ninth Lantern*

```
Characters
  Mira Fenn               (paragraph)
  Aldous Crane            (paragraph)
  Sella Vale              (paragraph)
  Toft                    (paragraph)
  Bryn Crane              (paragraph)
```

The bodies are, again, deliberately thin — a line of who-they-are, and the relationships the plot turns on. This is where voice notes, secrets, and the chapters a character enters and leaves will eventually live, but on day one a sentence each is right:

*Editor · Characters / Mira Fenn — the entry body*

```
= Mira Fenn

Protagonist and POV. Young under-keeper, apprenticed
to the harbourmaster; practical, stubborn. The one
who finds the cold lantern and goes looking.
```

Note one thing we are **not** writing yet: nowhere in Aldous Crane's entry does it say he put the ninth lantern out himself. That is the book's central secret, and who is allowed to know it — and when — is a whole later chapter's concern. For now the roster just needs to exist. Recording a name is not the same as spending its secrets.

FICTION

Your Characters book is the cast list and your Places book is the map's index. The five names here are what will, in later chapters, switch on continuity, voice, and the who-knows-what reader — all from these paragraphs.

NON-FICTION

For a non-fiction book the same two books hold recurring **entities**: the people your argument keeps naming and the institutions or sites it returns to. The overlay then doubles as a spell-check — a name written two ways stops lighting up on the variant.

## A light touch of world.hjson

Places and Characters record **what things are called**. There is a third, optional file that records **how the world behaves** — its physics — and Saltmarch needs a whisper of it: the coast is cold, and the fret comes off the water. That file is `world.hjson` at the project root. It is opt-in — without it nothing changes — and once you write one and hand it to the project (two commands, just below), the world layer wakes up.

We are going to keep this to the bone. Only two blocks are load-bearing — the world's `name` and its `astronomy` (everything about seasons and tides derives from the sky) — and then one small `geography` landmark to pin Saltmarch's cold. That is all the story wants.

*world.hjson — Saltmarch, kept small*

```
{
  name: "Saltmarch coast"
  seed: 0x5A17
  primary_language: "en"

  astronomy: {                 // the only required block
    star:   { class: "G2V", luminosity_solar: 1.0 }
    planet: { axial_tilt_deg: 23.4, day_length_hours: 24.0 }
    orbit:  { semi_major_axis_au: 1.05, year_length_days: 365 }
    moons:  [ { name: "the Watch", period_days: 29.5 } ]
    calendar: { months: 12, month_length_days: 30, weekdays: 7 }
  }

  geography: {
    landmarks: [
      { name: "Saltmarch", kind: "town",
        climate_zone: "subarctic", population: 1400 }
    ]
  }
}
```

Two small faithfulness points earn a mention. First, the **HJSON quoting rule**: an unquoted string runs to the end of the line, so an inline enum must be quoted — `class: "G2V"`, `kind: "town"`, never bare. When a value comes out looking wrong, this is nearly always why. Second, the landmark's `climate_zone` turns Saltmarch into a **gazetteer** entry: the fact-checker now knows the town's weather by name, so a chapter that puts a warm afternoon on this cold coast has something to be measured against. We push the orbit out to `1.05` AU and give the world a subarctic town precisely so "cold" is a declared fact, not a vibe.

### Handing it to the project

Writing the file is not the last step — the project has to be told to read it. Two commands do that. `inkhaven realworld validate` parses `world.hjson` and checks that every layer actually **compiles**, so a stray quote or an impossible orbit surfaces here rather than three chapters later. Then `inkhaven realworld compile --layer astronomy --materialize` derives the seasons, tides, and calendar from the sky you declared and **writes them into the World system book** — a real, editable chapter in your tree. That last command is the import: after it, the world is a part of the project, and the fact-checker has the seasons and the gazetteer to measure a scene against.

*Validate, then materialize the sky into the World book*

```
$ inkhaven realworld validate
ok — world `Saltmarch coast`, seed 0x5a17, primary language `en`
  astronomy:    ok · 1 moon(s), 12-month calendar
  geology:      ok · 3 plate(s), 2 continent(s)
  climate:      ok · 5 biome(s)
  hydrology:    ok · 4 river(s), 1 lake(s)
  demographics: ok · 7 settlement(s)

$ inkhaven realworld compile --layer astronomy --materialize
astronomy · Saltmarch coast
  year:     365.0 planet-days  (365.2 Earth-days, 1.000 M☉ star)
  tilt:     23.4°
  moon the Watch:  synodic 29.5 planet-days, 12.4 lunations/yr
  tides:    the Watch dominant; sun 0.46× the dominant moon
  → World/Astronomy: 6 paragraph(s) created, 0 updated
```

**MATERIALIZE THE SKY, DO NOT SIMULATE A WORLD**

Compiling `--layer astronomy` is cheap and honest: it writes back only the sky you declared. The full compiler goes much further — deriving whole continents, rivers, and cities from the seed — and that is a large feature with a companion book of its own. **The Ninth Lantern** needs none of it. We declare the sky and one cold town, materialize that much, and stop. Reach for `--layer all` when a story's geography has to hold together across a map; a single harbour town does not.

## Watching it light up

Now the reason we did all this before drafting. Open any paragraph of the manuscript and Inkhaven compiles a lexicon from the two books' titles and lays it over your prose: every word that matches a Place name renders **cyan and bold**, every word that matches a Character name renders **yellow and bold**. No tagging, no markup — your world becomes visible in the sentence as you write it.

A monospace page cannot show colour, so picture the overlay as a highlight over the matching words. Here is the opening line we will actually write next chapter, seen through it:

● ● ● *A draft line, mentions lit by the overlay*

```
Editor · The Ninth Lantern / ch.01 / 01-cold-morning

Mira ran the length of the Long Mole to where the
ninth lantern stood dark, and behind her all
Saltmarch still slept.

lit cyan   → the Long Mole · the ninth lantern
             · Saltmarch          (Places)
lit yellow → Mira                 (Character)
```

Everything lit up on the first line we ever showed the tool. Three things worth naming happened there:

- **Stemming.** We recorded `Mira Fenn`, but the prose here says just `Mira`, and a possessive `Mira's` two paragraphs later will light too — the matcher compares **stems**, keyed to the project `language` (English by default), not letters. This is the same machinery that makes a Russian `Москве` light an entry titled `Москва`.
- **Multi-word titles** match as a **run** of stems. `The Long Mole` lights the whole phrase, not a stray `Long`, and only where the run appears together.
- **A collision rule.** If a name were both a Place and a Character, **Place wins by design**. Watch for it with surnames that are also place names; `Crane` is a family here, not a location, so we are safe.

The overlay refreshes live as you type, again on every `Ctrl+S`, and once more from scratch on project open — so it is always current. Add `Bryn Crane` to the Characters book at noon and every `Bryn` in the manuscript is yellow by the time you save.

> **THE OVERLAY IS PASSIVE**
>
> It shows what you recorded; it never talks back, and it never edits your prose. A name that **should** light but does not is the fastest continuity check there is — it means the entry is missing or spelled differently. Trust the dark word.

### Asking the assistant, grounded in our canon

The overlay is the passive half. The active half is the same two chords, used to **ask**. Put the cursor in `Saltmarch` in your prose and press `Ctrl+B P`. Inkhaven sweeps the Places book for every paragraph whose title contains your term, builds

a context block from their bodies, and — because our AI prompt bar is empty — **arms** it and jumps focus to the prompt. The status line reads:

● ● ● *Status line after Ctrl+B P on `Saltmarch`*

```
Place RAG armed for `Saltmarch` — type your question
and Enter
```

Type a question — "what does the town believe the lanterns are for?" — press Enter, and the model answers from **our** text, not from whatever it half-knows about real salt marshes. The block it receives is exactly our entry, wrapped:

● ● ● *The context block prepended to the question*

```
── Place context for `Saltmarch` (1 match(es)) ──

── Place: Saltmarch ──
= Saltmarch
A poor fishing town on a cold northern coast. Stone
quays, a working harbour, sea-fret off the water most
mornings. Nine lanterns ring the harbour; the creed
is that no lantern goes dark.
── end place ──
```

`Ctrl+B C` does the identical thing against the Characters book — cursor in `Aldous`, ask "what does he know that the others do not?", and the model reasons from Aldous's recorded entry. (Had the prompt bar already held text, the chord would fire immediately instead of arming.) This is how you keep an assistant inside a canon it is otherwise happy to invent around: give it the entry as ground, and ask.

**FICTION**

`Ctrl+B P` / `Ctrl+B C` keep the AI answering from your bible in miniature — the town's belief, a keeper's secret — instead of a plausible-sounding invention that quietly contradicts chapter nine.

**NON-FICTION**

The same chords ground the assistant in a non-fiction **entity's** recorded facts — an institution's founding date, a figure's real title — so an answer cites the ledger you verified rather than the model's training-time guess.

**WHAT YOU LEARNED**

- The **Places** and **Characters** system books are seeded at `init`; you record an entity as a paragraph whose **title is its name**, and saving is registering — no separate step.
- We gave **The Ninth Lantern** a small map (Saltmarch, the harbour, the Long Mole, the nine and the ninth lanterns) and a five-name cast (Mira, Aldous, Sella, Toft, Bryn), each body kept deliberately thin — and left the central secret unwritten.
- A light `world.hjson` — `name` + `astronomy` + one `geography` landmark — declares the cold coast; `realworld validate` then `compile --layer astronomy --materialize` hands it to the project (into the World book) so "cold" is a checkable fact, without simulating a whole world.
- Mentions light up live in the editor — **cyan** Places, **yellow** Characters — with **stemming** keyed to the project `language`, multi-word titles matched as a run, and Place winning any collision.
- `Ctrl+B P` / `Ctrl+B C` sweep the matching book and **arm or fire** a question grounded in your own canon; the same act of filling these books switches on the continuity, voice, and knowledge readers we meet later.

PART II

# Drafting

CHAPTER 3

# 3

# Writing the First Chapters

The world is sketched and the cast has names (Chapter 2), which means the part of the work no tool can do for you is finally due: putting one word after another. This chapter is a single morning of it. We draft the opening of **The Ninth Lantern** — Mira Fenn walking out to the end of the Long Mole in the fog and finding the ninth lantern cold — and, as we go, we meet the three surfaces you will live inside for the whole draft: the **Editor** where the prose lands, the **Search bar** that finds a line again a week later, and the **AI assistant** you can lean on for a stubborn sentence. None of them writes the book. They keep the writing frictionless, and they get out of the way.

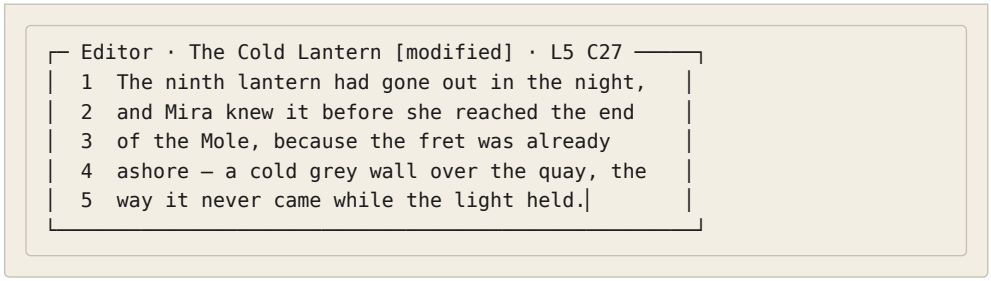
## One paragraph, open in front of you

You start where every scene starts: an empty paragraph. In the Tree you move the cursor to the slot that will hold the opening — call it **The Cold Lantern** — and press `Enter`. Focus jumps to the Editor, the (still empty) `.typ` file loads, and a line-number gutter appears down the left. From here you type.

> **THE EDITOR HOLDS ONE PARAGRAPH**
>
> The Editor is not a scroll of the whole chapter — it holds a **single paragraph**, one `.typ` file, because a paragraph is Inkhaven's unit of prose, of snapshot, and of search. You move **between** paragraphs in the Tree; you write **within** one here. Opening the next one saves this one first, so the boundary costs you nothing.

An hour in, the opening looks like this:

● ● ● *The Editor — the opening of The Ninth Lantern taking shape*




The pane is a running readout of where the words stand. The **border colour** speaks while the Editor has focus: **green** means the buffer matches the file on disk, **yellow** means you have unsaved edits. Unfocused, the border goes plain and hands the signal to two always-on marks — the `[modified]` chip in the title and the red ● in the status bar — so you can tell a dirty paragraph from anywhere in the window. The title also carries a live cursor read-out, `L<row> C<col>`, the quiet coordinate you glance at when a reading tells you to "jump to line 40."

Moving around is entirely conventional, which is the point — your fingers already know it. Arrows move a cell or a line; `Ctrl+←` and `Ctrl+→` jump by word; `Home` and `End` snap to the ends of the line; `Ctrl+Home` and `Ctrl+End` leap to

the top and bottom of the paragraph. When a scene runs several beats separated by a break line — `* * *` between them — `Ctrl+B >` hops the cursor forward to the next break and `Ctrl+B <` back to the previous one, so you walk the beats without hunting for them. There is nothing here to unlearn.

## Watching the draft change

Here is the first small kindness the Editor does you, and you will come to rely on it without noticing. As you write, Inkhaven renders in **bold** every character you have added since the paragraph was last saved. It is a running visual diff against the version on disk: at a glance you see exactly what **this** pass has touched. Say you had saved after "…the fret was already ashore," and then, still picking at the sentence, you kept going. The clause you just added stands out:

● ● ● *Bold marks what is new since the last save*

```
┌─ Editor · The Cold Lantern [modified] · L5 C27 ──────┐
│  1  The ninth lantern had gone out in the night,     │
│  2  and Mira knew it before she reached the end      │
│  3  of the Mole, because the fret was already        │
│  4  ashore — a cold grey wall over the quay, the     │
│  5  way it never came while the light held.          │
│                                                      │
│  bold on screen (new since save): from the em        │
│  dash on line 4 to the cursor.                       │
└──────────────────────────────────────────────────────┘
```

The bolding **clears the moment you save** — bold means "new since disk," and a save makes buffer and disk agree, so there is nothing left to mark. Watching the bold dissolve on `Ctrl+S` is the simplest confirmation your work has landed.

You will reach for `Ctrl+S` less than you expect, because Inkhaven also saves for you at three natural moments: after a few seconds idle, when you open another paragraph, and the instant focus leaves the Editor. Moving around the window therefore costs you no work — the deliberate save is there for the moments you want the confirmation. And every save does one more thing you will cash in shortly: it re-embeds the paragraph into the semantic index, so a line you write this morning is **findable** this afternoon.

## A snapshot before a risky change

You reread the opening and the first line nags. **“The ninth lantern had gone out in the night”** is fine, but you wonder whether starting on the fog — the thing the reader can feel — lands harder. This is a real rewrite, the kind you might regret, so before you touch it you take a **snapshot**.

> **TERM Snapshot**
>
> A **snapshot** is a point-in-time bookmark of a paragraph. `F5` takes a fresh one; `F6` opens the picker of every snapshot for this paragraph, to load, compare, or restore. Snapshots are how you experiment without fear: capture the version you have, try something bolder, and if the bolder thing is worse, the earlier one is one keystroke away.

Press `F5` and the current opening is safe. Now you rewrite the first two lines outright. To see where you came from while you work, `F4` splits the Editor: your live, editable buffer sits **above**, and a frozen copy of the pre-`F4` version sits **below** in dim grey.

● ● ● *F4 split-edit — the new opening above, the old frozen below*

```
┌─ Editor · The Cold Lantern [modified] ──────────┐
│  1  The fret was ashore before the light failed —  │
│  2  Mira felt it on the Mole, and knew.|           │
├─ snapshot · line 1/5 · Ctrl+H/J scroll ─────────┤
│  1  The ninth lantern had gone out in the night,   │
│  2  and Mira knew it before she reached the end    │
└─────────────────────────────────────────────────┘
```

You read the two side by side and decide the new version is too clever — it gives away the fog’s menace before the lantern has even earned it. So you throw it back: `Ctrl+F4` **accepts the snapshot**, replacing the live buffer with the frozen original wholesale. (Had you liked the rewrite, you would simply have pressed `F4` again to close the split and keep it.) Nothing was lost either way, because you bookmarked before you leapt — and even without the split, `F6` would have offered that same

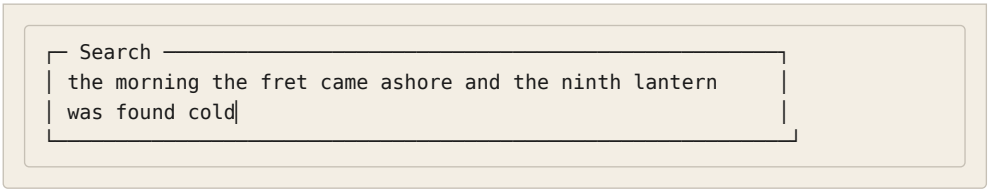
snapshot back. This is the habit worth building: `F5` before any change you are not sure of, and experiment freely.

## Finding that line about the fog, later

A week of drafting later, you are three chapters on and you want **that line about the fog rolling in cold off the water** — you can feel the shape of it, but you have written a dozen foggy mornings since and you cannot remember which paragraph holds this one, or how you actually phrased it. This is the job semantic search was built for, and it is one keystroke away: `Ctrl+/` (or `Ctrl+4`) focuses the Search bar.

You do not type keywords. You **describe the passage** — the fuller the query, the sharper the result:

● ● ● *A query by feeling, not by exact words*




Press `Enter`, and a ranked overlay drops down — the nearest paragraphs across the **whole** project, best match first:

● ● ● *Results ranked by meaning — the prose, and the note that fed it*

```
┌─ Results · "…the ninth lantern was found cold" (4) ──────────┐
│  0.912  [paragraph] Lantern › One › the-cold-lantern         │
│         The Cold Lantern                                     │
│         The ninth lantern had gone out in the night…         │
│                                                              │
│  0.861  [paragraph] Lantern › One › out-along-the-mole       │
│         Out Along the Mole                                   │
│         She went past the eighth pillar, where the fret…     │
│                                                              │
│  0.834  [note]      Notes › the-sea-fret                     │
│         What the fret is                                     │
│         A cold fog off the deep water; the town believes…    │
├──────────────────────────────────────────────────────────────┤
│ ↑↓ select · Enter open · Esc close                           │
└──────────────────────────────────────────────────────────────┘
```

Look at what just happened. Your query said **fret came ashore**; the top paragraph says **"had gone out in the night."** Your query said **found cold**; the paragraph names no such thing. The words are strangers, yet it is unmistakably the passage you meant — because semantic search matches by **meaning**, not by characters, and the meanings are neighbours. It even surfaces the worldbuilding note on the sea-fret you took days ago, alongside the scene it informed, because both were indexed the same way. `↑` and `↓` move the selection; `Enter` on a hit **opens that paragraph in the Editor** and repositions the Tree onto it, so one keystroke takes you from "the scene where the fog comes in" to the cursor blinking inside it.

### SEMANTIC SEARCH, NOT LITERAL — KNOW WHICH YOU WANT

The Search bar finds passages by **sense**. When instead you want a **specific string** — a character's exact name, a coined term, a phrase you are hunting to replace — that is a different tool: `Ctrl+F` runs a literal regex find inside the open paragraph, and `rg` over your `books/` folder searches the files on disk. Meaning-based for "the passage about a thing"; literal for "every place I typed these exact letters."

## Steering the assistant to tighten a line

There is a third party you can call to the desk without leaving the window: a language model in the AI pane. Inkhaven's stance on it is one line — **AI is a co-author you steer** — and every part of the pane is built around that word. The assistant never acts on its own, never sees more of your book than you hand it, and **never changes a word of your prose until you press the key that says so.**

Back in the opening paragraph, one sentence still sags. **"— a cold grey wall over the quay, the way it never came while the light held"** is two ideas wearing one sentence. You could worry it yourself; this once, you ask for a second pair of eyes. Steering the model is three small dials.

First, **scope** — what the model is allowed to see. By default it sees only your prompt; you attach a slice of the manuscript with `F9`, which cycles a ring of scopes. You select the sagging sentence in the Editor (`Shift`+arrows), then press `F9` until the pane reads `scope=Selection` — now the next prompt carries exactly that sentence and nothing more.

Second, **mode** — how much of its own training the model may lean on. `F10` toggles between `Full` (its general knowledge is fair game) and `Local` (use **only** what I have shown you). For "tighten **this** sentence," you want `Local`: the job is to reshape your words, not to reach for someone else's.

Then you type. `Ctrl+I` drops focus to the prompt bar; you ask plainly and press `Enter`. The answer streams in live — you watch it arrive:

● ● ● *A scoped, local-mode ask — the model reshaping your own words*

```
┌─ AI · ollama · streaming… · infer=Local · scope=Selection ──┐
│  you  Tighten this to one clean line. Keep the word "cold".  │
│  ai   The fret was already ashore, cold off the deep         │
│       water, and it never came while the light               │
└──────────────────────────────────────────────────────────────┘
```

When it finishes, the title flips to `done` and a row of **destination chips** appears in the footer. Focus is still on the prompt bar — handy for a follow-up question — but the chip keys are live only **inside the AI pane**. Press `Esc` to step from the prompt bar into the pane. Nothing has touched your buffer yet — the answer is a proposal, inert until you choose:

● ● ● *Done — the scope has reset, and the proposal waits for a key*

```
┌─ AI · ollama · done · infer=Local ──────────────────────────┐
│  ai  The fret was already ashore, cold off the deep         │
│      water, and it never came while the light held.         │
│                                                             │
│  r replace · i insert · t top · b bottom · c copy           │
└─────────────────────────────────────────────────────────────┘
```

You read it against your own. It is tighter, it keeps **cold**, and it is genuinely better — so you accept it. Now in the pane, `r` **replaces the selection** with the model's text, converting its markdown to Typst on the way in, and jumps focus back to the Editor so you land where the change did. And here the two threads of this chapter meet: because an apply marks the buffer dirty, the new sentence arrives **bold** — the same "new since save" diff you watched earlier — so you can read exactly what changed against what is still on disk:

● ● ● *Back in the Editor — the accepted line, bold as a change you can still undo*

```
┌─ Editor · The Cold Lantern [modified] · L4 C52 ──────┐
│  1  The ninth lantern had gone out in the night,     │
│  2  and Mira knew it before she reached the end      │
│  3  of the Mole, because the fret was already        │
│  4  ashore, cold off the deep water, and it never    │
│  5  came while the light held.                       │
│                                                      │
│  bold on screen (new since save): the replaced       │
│  sentence — Ctrl+U to revert, Ctrl+S to keep.        │
└──────────────────────────────────────────────────────┘
```

That is the whole contract. Had the proposal been wrong, `Ctrl+B C` would have cleared it unapplied, or you would simply have never pressed `r`; and even after accepting, `Ctrl+U` walks the change straight back out — the edit is yours to approve or reject, always. The model advised; you decided. Note too that `Local` mode governs only what the model **draws on**, not where the call runs — on an external provider your prompt still travels to their servers; if you want the whole exchange to stay on your machine, that is a **provider** choice (run Ollama), which is exactly what the pane above is doing.

### THE ASSISTANT ADVISES; YOU HOLD THE KEY

This is the one habit to internalise about AI in Inkhaven, and it holds everywhere the assistant touches prose: the pane only ever **shows** you something. A separate, deliberate keystroke — `r`, `i`, `t`, `b`, or the `g` of a grammar-check — is what lets it land, and the change then arrives as a visible diff you can undo. No word of your manuscript ever changes behind your back.

### FICTION

Reach for the assistant the way you just did: a **scoped, local** ask to reshape a line you have already written — tighten a sentence, vary a repeated verb, cut a clause. It is a sharpening stone for your own prose, never a ghostwriter; the words that land are the ones you accept.

### NON-FICTION

The same move serves an argument: select a clause and ask, in `Local` mode, to make it **precise** rather than merely shorter — collapse a hedged claim into one clean statement, or surface a buried qualifier. You steer, you read the diff, you accept only what sharpens the point.

**WHAT YOU LEARNED**

- The **Editor** holds **one paragraph** — a plain `.typ` file — and shows its save state three ways: a green (saved) or yellow (dirty) focused border, the `[modified]` title chip, and the red ● in the status bar. Movement is conventional; `Ctrl+B >` / `Ctrl+B <` hop between scene breaks.
- Inkhaven renders in **bold** every character added since the last save — a running diff against disk that clears the moment you `Ctrl+S`. It also autosaves on idle, on paragraph switch, and on focus loss, re-embedding as it goes.
- Before a risky change, `F5` takes a **snapshot**; `F4` splits the Editor to edit against the frozen version (`Ctrl+F4` accepts it back), and `F6` reopens any snapshot later. Experiment freely — the earlier version is one key away.
- `Ctrl+/` (or `Ctrl+4`) opens **semantic search**: describe the passage by feeling and it finds the paragraph even when your query shares no words with it. Use `Ctrl+F` or `rg` instead when you want a specific literal string.
- The **AI assistant** is a co-author you steer with three dials: `F9` sets **scope** (what it sees — here `Selection`), `F10` sets **mode** (`Local` = only your context), and — once you `Esc` from the prompt into the pane — a **destination chip** (`r` to replace) lands the answer. It advises in the pane and changes nothing until you press the key — and the result arrives as a bold, undoable diff.

CHAPTER 4

# 4

# Keeping the Facts Straight

By the third chapter of **The Ninth Lantern** there were already more small truths than one head holds. Nine lanterns — not eight, not ten. The ninth, the cold one, alone at the far end of the Long Mole, not among the ring along the quays. The fret coming ashore at **dawn**, not at dusk. None of these is hard to remember on the morning you invent it. Every one of them is easy to contradict on a tired afternoon four chapters later, when the rhythm of a sentence wants a different number and your hand writes "the eighth lantern" without your noticing.

These are not mistakes of craft. The sentence that says "the eighth lantern was cold" is a perfectly good sentence; it is only wrong about the book it lives in. And it is exactly the kind of wrong you cannot catch by rereading, because you know

what you **meant**, and your eye supplies it. Inkhaven's answer is to stop trusting your memory: write the settled truths down in a place the tool can read, and then let two different readers hold your prose to them. This chapter meets both — the one you invoke when you want a paragraph checked, and the one that reads over your shoulder as you type.

> **TWO DIFFERENT READERS**
>
> The **fact-check** (`Ctrl+B Shift+X`) is a reader you **ask**: it weighs one paragraph against the truths you wrote in the Facts book, and streams back a verdict. The **continuity watch** is a reader that never sleeps: turn it on and it re-checks continuity on every save, deterministically, for free. The first is about **what is true**; the second is about **what stays consistent**. You want both, and they do not overlap.

## Writing down what's true

In the last chapter you sketched the world in `world.hjson` — the harbour, the coast, the physics of the place. That file records what the world **is**. The truths that a simulation could never derive — that there are nine lanterns and not eight, that the ninth is the one that went dark, that the keepers' creed is **no lantern goes dark** — live somewhere else: the **Facts book**.

> **TERM** **The Facts book**
>
> A system book of the settled truths of your story, written as ordinary paragraphs in plain prose. It sits beside your manuscript, never inside it. Distinct from `world.hjson`: the world file **derives** a world from physics; the Facts book **records** the decisions you have made and mean to keep — the ones no simulation could produce.

You write into it exactly as you write anywhere else: open the Facts book, create a paragraph, type the truth. Keep each one short and flat — one settled fact per paragraph reads best, both for you and for the checker that will parse it. Here are the three facts **The Ninth Lantern** cannot do without.

*Three entries in the Facts book*

```
Facts / The lanterns
──────────────────────────────────────────────

There are nine lanterns. They ring the harbour
on stone pillars; the ninth stands alone at the
end of the Long Mole.

No lantern has gone dark in three hundred years.
The keepers' creed is: no lantern goes dark.

The sea-fret comes off the water at dawn. The
light is what holds it back from coming ashore.
```

That is the whole ritual. There is no schema to satisfy and no field to fill — the Facts book is prose, and it stays readable as prose. What it buys you is that from this point on, every claim your manuscript makes about lanterns, about the age of the light, about the fret, has something concrete to be measured against. Two facilities read it. One you meet now.

> **THE FACTS AI SCOPE**
>
> The same entries also ground the assistant. `F9` cycles the AI scope, and **Facts** is one of the sticky scopes: select it once and every follow-up is answered from your written truths rather than freshly invented ones — so the AI never quietly retcons the harbour mid-conversation. For a large Facts book, `Ctrl+B Shift+S` opens a semantic search over it, so you ground in the handful of relevant facts instead of the whole book.

## The fact-check — Ctrl+B Shift+X

A few pages into a new scene, Mira is out on the quay in the grey before sunrise, and you write the line that names what is wrong:

*The line you just wrote*

```
She had counted them twice on the way out. Eight
lanterns burning down the harbour, and the eighth
one — the one at the end of the Mole — stone cold.
```

Everything about that sentence sounds right. It has the count, the place, the cold. It is also wrong twice over: the lantern on the Mole is the **ninth**, and if the ninth is dark then **eight** are burning, not counting a cold one among them. You will not see it. You wrote it. So you ask.

With the cursor in that paragraph, `Ctrl+B Shift+X` runs the fact-check. It locks the AI scope to this one paragraph, grounds the check against every entry in the Facts book — the lanterns, the creed, the fret — and streams a verdict into the AI pane, flagging any claim that contradicts what you wrote down. Its mnemonic is **X** for fact e**X**amination.

*The verdict streams into the AI pane*

```
┌─ AI · fact-check · this paragraph ──────────────────┐
│ ⊗ Contradiction                                     │
│   You wrote: "the eighth one — the one at the end   │
│   of the Mole."                                     │
│   Recorded fact: the lantern at the end of the Long │
│   Mole is the NINTH. The one that went dark is the  │
│   ninth, not the eighth.                            │
│                                                     │
│ ⚠ Also check: "Eight lanterns burning." If the cold │
│   lantern is the ninth, eight burn and one is dark. │
├─────────────────────────────────────────────────────┤
│  Ctrl+B Shift+J  jump to the next flagged claim     │
└─────────────────────────────────────────────────────┘
```

The check found what your eye slid over. Now you fix it — and here the second chord earns its place. After a fact-check flags contradictions, `Ctrl+B Shift+J` cycles the editor cursor through them one at a time: each press jumps to the next flagged claim in the paragraph and shows the violated fact, with its explanation, on the status bar. Its mnemonic is **J** for **jump**. You walk the findings, you correct “eighth” to “ninth” and “eight lanterns” to “nine,” and the paragraph now agrees with the book it lives in.

> **WHEN THE FACTS BOOK IS EMPTY**
>
> The fact-check never fails for want of facts. With an empty Facts book it degrades gracefully to a generic local fact-check of the paragraph on its own terms, rather than refusing. And it works in all five of Inkhaven's languages — English, Russian, German, French, Spanish — detecting the paragraph's language and rendering the verdict in it. The honest question, **does this work in Russian?**, answers yes.

There is a command-line form for when you would rather sweep the whole book than check one paragraph: `inkhaven facts scan` walks every chapter, semantically matches each against the Facts book, and reports the contradictions — the scriptable, CI-friendly twin of the in-editor check (add `--json` for a machine-readable report). In the editor, though, `Ctrl+B Shift+X` on the open paragraph is the motion you will reach for, because it lands where you are already looking.

> **TWO COMMANDS, EASY TO CONFUSE**
>
> `inkhaven facts scan` — the one here — weighs your prose against the **Facts book**. Do not mistake it for `inkhaven fact-check`, which weighs prose against the **simulated world** from `world.hjson` (travel times, the gazetteer, the `magic:` ledger). Same instinct, different canon: one guards the truths you wrote down, the other the physics you declared.

## The book that watches as you write

The fact-check is a reader you summon. The second reader you switch on once and then forget, because it does its work in the same breath you make the mistake. Inkhaven's continuity intelligence — the layer that watches for characters in two places at once, for numbers that drift, for facts that quietly change across chapters — can be told to re-check continuity on **every save**. It is off by default, because not every author wants it; you turn it on in your config.

*Turning the watch on — the continuity block*

```
continuity: {
  enabled: true             // the review-pass ledger
  ambient: true             // re-check on every save  ←
  ambient_cooldown_secs: 30 // throttle floor (seconds)
  co_location: true         // per-detector toggles
  numeric: true
  char_facts: true
  introduce: true
}
```

The single line that matters is `ambient: true`. With it set, every time you save, the watch reads the paragraph you just touched — its entities, its chapter — re-runs the deterministic continuity detectors against just that scope, and surfaces anything new in the Output pane. Because the core is **deterministic and free** — it computes its answer from structure, with no model in the loop — this happens inline, with no background job and no pause you would notice, however long the manuscript has grown.

Suppose that, a chapter on, you describe Aldous's walk out to the cold lantern and write the Mole as **half a mile** of wet stone — forgetting that in Chapter 1 you called it **a quarter-mile**. Neither number is in the Facts book; this is not a truth you declared, just a measurement you let drift. The fact-check would say nothing. The watch does not miss it.

*The watch flags the slip on save*

```
┌─ Output · continuity watch ──────────────────────────┐
│ ⚠ [continuity · numeric]                             │
│   The Long Mole is "half a mile" here, but "a        │
│   quarter-mile" in ch. 1 — a distance that           │
│   conflicts across chapters.                         │
├──────────────────────────────────────────────────────┤
│  continuity watch: 1 finding(s) touching this edit   │
└──────────────────────────────────────────────────────┘
```

The status line tells you the edit touched something; the Output pane names the break, the detector that raised it (`numeric`), and both readings so you can see which to keep. Its sibling detector, `co_location`, would have fired the same way had you

put Mira on the quay and out on the Mole in the same grey morning — a character in two places at once. Neither costs anything, and neither waited to be asked.

> **A THROTTLE, NOT A QUEUE**
>
> A burst of rapid saves does not re-check on every keystroke-flush. The watch throttles itself with `ambient_cooldown_secs` (default 30) — a floor that collapses a flurry of saves into one re-check, rather than running the check that many times. Re-checking a paragraph replaces its own prior findings, so the Output pane never accretes stale warnings.

The watch is the **ambient** face of a larger machinery. When you want the whole book swept at once rather than the last edit's neighbourhood, `Ctrl+B Shift+I` opens the full continuity ledger — every deterministic finding, ranked and grouped, `Enter` to jump to each — and `inkhaven continuity check` runs the same sweep from the command line, exiting non-zero on any hard contradiction so it can gate a commit hook. The ambient watch is that same reader, narrowed to what your fingers just changed, running while you write.

## Two checks, two questions

It is worth keeping the pair distinct, because they answer different questions and you will reach for them at different moments. The fact-check asks **is this true?** — it weighs a claim against the truths you deliberately wrote into the Facts book, and it is a language-aware model reading for meaning, which is why it caught "the eighth lantern" as a matter of **fact**. The continuity watch asks **does this stay consistent?** — it is a deterministic structural reader, no model involved, catching the Mole that shrinks and the character who bilocates, whether or not you ever declared a fact about either. One guards the world you decided on; the other guards the book from itself.

**FICTION**

In **The Ninth Lantern** the Facts book is the series bible in miniature: nine lanterns, the ninth on the Mole, the fret at dawn. Write the truths the story turns on, and the fact-check holds every scene to them while the watch keeps the geography and the numbers from drifting underneath you.

**NON-FICTION**

In **non-fiction** the Facts book is the ledger of claims you have verified — dates, figures, definitions, the settled results of your research. The same `Ctrl+B Shift+X` holds each paragraph to your verified record, keeping the manuscript honest to your own evidence rather than to a half-remembered draft.

Between them, the two readers close the gap that memory cannot: one you ask when a paragraph feels load-bearing, one that answers unasked the moment a save lands. The lanterns will stay nine. In the next chapter the manuscript grows a secret — and a third reader that guards not what is **true**, but who is allowed to **know** it.

**WHAT YOU LEARNED**

- The **Facts book** records the settled truths of your story as ordinary paragraphs — the decisions no `world.hjson` simulation could derive (nine lanterns, the ninth on the Long Mole, the fret at dawn). It also grounds the AI through the sticky `F9` **Facts** scope.
- `Ctrl+B Shift+X` **fact-checks** the open paragraph against the Facts book and streams a verdict into the AI pane — it caught "the eighth lantern" as a contradiction of the recorded ninth. `Ctrl+B Shift+J` walks the flagged claims one at a time. It degrades gracefully with an empty Facts book and works in five languages.
- The **continuity watch** — `continuity.ambient: true` — re-checks continuity on every save over just the edit's scope, deterministically and for free, and surfaces breaks (`numeric` drift, `co_location`) in the Output pane inline. A `ambient_cooldown_secs` floor throttles it.
- The two are complementary: the fact-check asks **is this true?** against truths you declared; the watch asks **does this stay consistent?** structurally. Sweep the whole book with `Ctrl+B Shift+I` or `inkhaven continuity check`.

PART III

# The Middle

CHAPTER 5

# 5

# The Secret and Who Knows It

**The Ninth Lantern** has a secret at its heart, and the whole book is built to protect it. On the cold morning the story opens, the ninth lantern is dark and its keeper is gone, and for four chapters every character — and every reader — is allowed to believe the same wrong things: that the oil ran out, that Toft the merchant failed a delivery, that old Aldous wandered off into the fret and was lost. The truth is worse and quieter. Aldous **put the lantern out himself**, and walked out along the Long Mole on purpose, because he had learned what the light was really holding back. That truth is the hinge of the book. It lands in Chapter 6, in the one scene built to carry it, and it must land **there** — not a page sooner, in nobody's mouth, in no one's thoughts.

This is a different kind of mistake from the ones the last chapter guarded against. A character standing in two places at once, a river that shrinks between chapters, a distance that does not add up — those are fractures in **where and when**, and a reader half-feels them. A secret spoken too early is a fracture in **who knows what**, and a reader feels it exactly: the small, fatal click of a mystery giving itself away. No sentence is wrong. Every line is clean prose. The break lives in the gap between a scene in Chapter 5 and the reveal in Chapter 6 — a gap no single reading holds in view, which is precisely why you will read straight past it and a first reader will not.

Inkhaven has a reader that holds exactly that gap in view. It is called **KEN**, and this chapter is the one moment in the whole book where the tool does something no general-purpose assistant can: it reconstructs, across the entire manuscript, **who could know what, and by when**, and it flags anyone who outruns their own knowledge.

## The one axis that breaks a mystery

KEN is the sibling of the continuity watch you met last chapter. That watch — SENTINEL — has one cardinal move: it flags a thing **named before it exists**, an entity referenced in Chapter 2 that the book does not introduce until Chapter 5. KEN takes that move and carries it one step further, from **does this thing exist yet** to **could this person know this yet**. Same forward walk through the book, same Unicode-aware matching of names and phrases in the prose, a new axis: knowledge.

> TERM **Ken**
>
> A character's **ken** is the range of what they know — the set of facts they have been let in on, up to a given point in the book. KEN the reader is named for it. The whole check is a forward walk that builds each character's ken chapter by chapter and flags any line that reaches past it.

What makes this the book's signature "only Inkhaven can do this" moment is that KEN never **guesses** what a character knows. It **derives** it, from four things Inkhaven already holds and a chat model never will: the timeline (who was present when), the events' participant lists, the cast in your Characters book, and the point of view each scene is written from. A generic AI asked "does anyone know the secret too early?" can only re-read the prose and form an opinion. KEN reads the **structure** — the ground truth you declared — and computes an answer that is the

same every run and costs almost nothing. It reasons only over what it can ground, and where it cannot ground a thing it stays silent rather than inventing a break.

**FICTION**

For a mystery this reader is close to the whole reason the timeline and the cast exist. Keep the participant lists honest and tag your secrets, and KEN catches the leaked confidence and the too-early deduction a whodunit lives or dies by — the errors beta readers find and you cannot.

**NON-FICTION**

The same guard serves an **argument**. A conclusion used before it is established — a proof that leans on the very thing it has not yet shown — is premature knowledge in a suit and tie. Declare the claim, mark where you establish it, and the guard flags the paragraph that uses it early.

## Declaring the secret

KEN starts from nothing. A project with no secrets and no timeline gives it nothing to check, so it does nothing and costs nothing — you pay for this reader only in proportion to the epistemic structure you actually declare. The first thing you declare is the secret itself.

You mark it with a tag on the paragraph that carries the truth — the sentence, in the manuscript, that states plainly what happened. In **The Ninth Lantern** that is one line in the Chapter 6 reveal, and the tag names the secret in the words the rest of the book will use to refer to it. The words after the colon are the **topic**: a short, plain handle KEN will hunt for in the prose.

*The tag that names the secret*

```
secret:Aldous put the lantern out
```

That is the whole declaration. The tag does two things at once. It tells KEN the topic **exists** — that "Aldous put the lantern out" is a thing a character might refer to — and it tells KEN this particular topic is a **secret**, which raises the stakes on anyone who reaches for it before they are allowed to. A secret is not merely a fact that

arrives late; it is a fact whose early appearance is a leak. We will see the difference bite in a moment.

> **WHERE THE TAG LIVES**
>
> KEN reads the tags on your **manuscript** paragraphs — the same place your `pov:` marks live — not the Facts book you built last chapter. The secret is also a fact about your world, and it is fine for it to sit in your Facts book too; but the tag KEN acts on rides the sentence in the prose that states it. Put `secret:` where the truth is told.

### Setting, viewing, and finding a tag

You have seen **what** to write; here is **where** you write it. A tag is not typed into the prose — it lives in a small picker attached to each paragraph, so the sentence in your manuscript stays clean. Open the paragraph that states the truth and press `Ctrl+B ]`. The picker lists every tag this paragraph already carries, and every tag anywhere in the project: press `A` to add a new one, type `secret:Aldous put the lantern out` at the prompt, and `T` applies it. The same picker is how you **read a paragraph's tags back** later — open it, `Ctrl+B ]`, and there they are.

● ● ● *Ctrl+B ] — the per-paragraph tag picker*

```
tags · ¶ "…and the ninth had never once gone cold."

[x] secret:Aldous put the lantern out
[ ] pov:Mira
[ ] reveals:the end of the Mole

Space select · T apply · A add new · D delete (project-wide)
```

Finding them again is the other half. `Ctrl+B }` opens the search-by-tag picker: choose a tag and it lists every paragraph that carries it, with a filter box to narrow the list, and Enter on a paragraph opens it in the editor. This is how you audit your own scaffolding — "show me every scene I marked `know:`" — before you trust KEN to reason over it.

*Ctrl+B } — search by tag*

```
search tags:  know:

  know:Aldous put the lantern out@Aldous   ← Enter
  know:Aldous put the lantern out@Sella

── paragraphs carrying know:…@Aldous ──────────────
  ch. 1 · The keeper's round            Enter → open
```

**ONE PICKER, EVERY TAG**

`Ctrl+B ]` and `Ctrl+B }` are not KEN-specific — they set and find **any** paragraph tag: the `secret:` and `know:` of this chapter, the `pov:` of the next, a `reveals:` on a scene, or a private tag of your own. Learn the two chords once and every tagged feature in the book uses them.

## Granting the knowledge — two honest ways

A secret nobody may know is no use; the point is that **some** people know it, and the reader is watching them not say so. So the second thing you declare is the **grant**: the earliest point at which a given character could know a given topic. KEN derives a grant two deterministic ways, and never any other.

**TERM** **Grant**

A **grant** is the earliest place in the book a character could know a topic — the moment their ken widens to include it. It comes from **presence** (they were in the room) or from a **declaration** (you tagged it). A use of the topic before the character's earliest grant is a break; a use at or after it is clean.

### Presence — they were in the room

The first source is free, and it comes straight from the timeline. A character in an event's participant list **knows what happened at that event**, from the moment of the scene that depicts it onward. The reveal in Chapter 6 is an event on your timeline; its participants are the people present when Mira learns the truth — Mira

herself, and Bryn, who followed her onto the Mole. Listing them as present is all it takes: from that scene forward, both are granted knowledge of the secret, with no tag at all.

The reveal's own event title is often terse — "The end of the Mole" — too vague for KEN to match against the prose. A `reveals:` tag on the reveal scene binds that terse event to the real topic, so presence grants the thing the book actually talks about rather than a chapter heading.

*The reveal scene — presence + a bound topic*

```
event   The end of the Mole   ch. 6
        present: Mira, Bryn

tag     reveals:Aldous put the lantern out
```

### Declaration — you say so, by name

The second source is a tag, for the knowledge that no event conveniently records. Three forms cover it:

*The know: tags*

```
know:<topic>          grants the scene's POV character
know:<topic>@<name>   grants a named character
secret:<topic>        marks a topic secret (and, like
                      know:, makes it a thing KEN tracks)
```

Aldous is the one who did the thing, so he has known it from the first page. A `know:Aldous put the lantern out@Aldous` tag on an early scene grants him at Chapter 1 — now his own later remembering of it, in a flashback or a found letter, reads clean instead of tripping the guard. And Sella, the harbourmaster, learns it only when Mira brings the truth back to her — a Chapter 6 scene. A `know:Aldous put the lantern out@Sella` there grants Sella at Chapter 6, and not before. Sella's ken, for this one topic, opens in Chapter 6. Hold that thought.

> **THE PRECISION LADDER**
>
> When more than one source could name a topic, KEN takes the most precise: an explicit `know:` or `reveals:` tag beats a bare event title. That is why the `reveals:` tag on the Mole scene matters — it hands KEN the exact words to match, instead of leaving it to guess from a terse event name.

## The slip — a chapter too early

Now the reason for all of it. Somewhere in the drafting of Chapter 5 — a chapter **before** the reveal — you wrote a scene on the quay where Sella, cornered and angry, says more than she should. It is a good line. It is also impossible:

*Chapter 5, on the quay — one line too knowing*

```
"Don't look for a spill or a dry wick," Sella
said. "Aldous put the lantern out himself. He
chose the dark, and he chose it for us."
```

Sella cannot say this. Her grant for the topic opens in Chapter 6, when Mira tells her. Here she is naming it in Chapter 5, in her own mouth, a full chapter ahead of the only moment that could have told her. You will not catch it — you know the ending, so every rehearsal of it sounds natural to you. Run the reader that does not know the ending:

*inkhaven knowledge — the deterministic check*

```
$ inkhaven knowledge
⊗ [premature_knowledge] Sella speaks of "Aldous
  put the lantern out" in ch. 5 — before learning
  it in ch. 6.

1 finding(s): 1 break(s), 0 other.
```

There it is: the click of the mystery giving itself away, caught before a reader ever sees it. KEN attributed the line to Sella — the dialogue-attribution layer knows

**"Sella said"** names the speaker — matched the secret's topic inside her speech, looked up her earliest grant, found it a chapter **later** than the line, and raised a `premature_knowledge` break: a character acting on a fact before they could have learned it. The glyph `⊗` is a hard break; the kind is in brackets; the message names the character, the topic, the chapter of the slip, and the chapter where the knowledge actually arrives.

> **IT IS A GATE, NOT JUST A REPORT**
>
> Like the continuity check, `inkhaven knowledge` **exits non-zero** whenever a hard break survives — `premature_knowledge` or `leaked_secret`. That single rule lets it sit in a pre-commit hook or a CI pipeline: the build fails on a leaked secret the way it fails on a compile error. Add `--json` and the same findings come back as an array for a script to read.

## When a fact is a secret

Watch what the `secret:` tag was quietly doing. Suppose, in an earlier draft, you had granted the knowledge and tracked the topic but **not** yet told KEN it was secret. The slip above is what you would get: `premature_knowledge`, a character who knows a fact too early. Serious, but ordinary — the same finding KEN raises when a detective names the murder a chapter before the body is found.

Now add the one tag that says **this fact is a secret**, and re-run. The break does not change chapters, and the message reads the same — but its **kind** does:

● ● ● *The same slip, once the fact is a secret*

```
$ inkhaven knowledge
⊗ [leaked_secret] Sella speaks of "Aldous put the
  lantern out" in ch. 5 — before learning it in
  ch. 6.

1 finding(s): 1 break(s), 0 other.
```

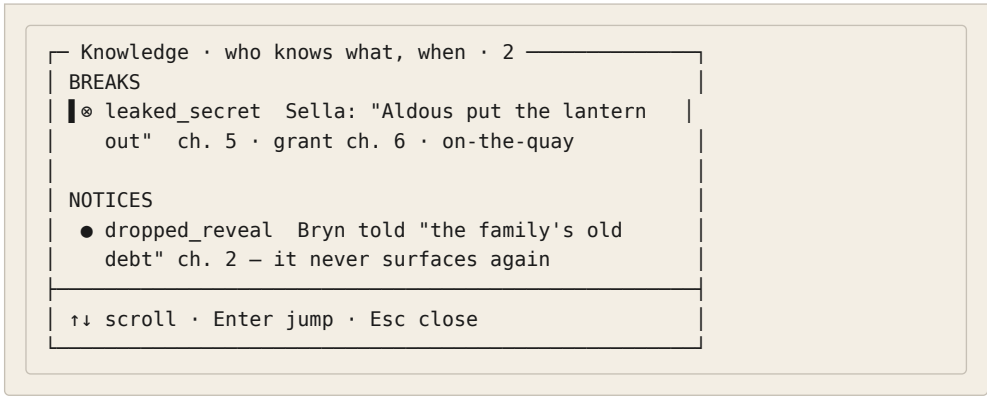
`leaked_secret`. KEN has raised the stakes on the identical line, because you told it the topic is not just late but **guarded**. This is the escalation worth carrying out of the chapter: a too-early use of an ordinary tracked topic is

`premature_knowledge`; a too-early use of a `secret:` topic is a `leaked_secret`. Both are hard breaks, both fail the gate, but the second says something sharper — **your book let its secret out a chapter early** — and it is a thing only a tool that knows which facts you meant to keep could possibly say.

## The dashboard — Ctrl+B Shift+Z

You do not have to leave the editor to see this. `Ctrl+B Shift+Z` opens the **knowledge dashboard** — every finding grouped by kind and ranked, breaks at the top. `↑↓` scrolls it, `Enter` jumps straight to the offending paragraph so you see the slip in its own context, and `Esc` closes it. It is the fastest way to walk a book's secrets without losing your place in the prose.

● ● ● *Ctrl+B Shift+Z — who knows what, when*




### dropped_reveal — the epistemic unpaid setup

That second line is a softer finding, and it repays a look. Bryn "knows more of the family's past than he says," so early on you tagged a scene `know:the family's old debt@Bryn` — granting him a piece of buried history. But in the drafting, that thread got away from you: the old debt is granted and then never spoken of, never acted on, never paid off. KEN notices. A declared reveal whose topic never surfaces again is a `dropped_reveal` — an epistemic **unpaid setup**, dangling knowledge you either forgot to use or a tag you can retire. It is a `●` notice, not a break; it does not fail the gate. It only asks the question you would want asked: **you told me Bryn knows this — did you mean to do anything with it?**

> **ONLY YOUR DECLARATIONS COUNT HERE**
>
> `dropped_reveal` fires only for knowledge you **declared** with a `know:` tag — the places you deliberately opted in. Knowledge granted by mere presence at an event does not have to resurface: a character can witness a storm and never mention it again without that being a loose thread. The dropped-reveal notice watches the promises you made on purpose.

## The fix — move the line, or grant the knowledge

Here is the part that matters most, and it is the part KEN refuses to do for you. It flags; it never rewrites. A knowledge break is not a phrasing to correct — it is a **choice about the story**, and there are two honest answers, each of which clears the finding:

- **Move the line.** Sella should not know yet, and the scene is stronger for her not knowing — so the revelation belongs later. Cut her too-knowing line from Chapter 5, or move the beat to after her Chapter 6 grant, and her ken no longer outruns her lips. The gate goes green.
- **Grant the knowledge.** On reflection Sella **should** have known all along — she is the keeper of the town's story, and perhaps she has known since before the book began. Then the fix is not to silence her but to make her knowledge true: give her an earlier grant, with a `know:Aldous put the lantern out@Sella` tag on an early scene, or by adding her to the participant list of an earlier event she was present at. Now the line in Chapter 5 sits **after** her grant, and it is no longer a leak — it is foreshadowing you meant.

Which of these is right is a decision no tool can make, because it is a decision about what your book is. That is exactly how Inkhaven routes it. When these findings reach the **Editorial Pass** — the revision worklist of a later chapter — a knowledge break arrives not as a mechanical rewrite but as a **decision**: **fix the leak, move the reveal, or add a grant?** You choose; the tool holds the consequences. And when you have chosen, re-run the check and watch the book go quiet:

*After the fix*

```
$ inkhaven knowledge
✓ no epistemic breaks — nobody knows what they
  shouldn't.
```

## The deep pass — for the slip with no words

Everything so far is deterministic: a forward walk, set membership, and matching a topic's **name** in the prose. It costs essentially nothing and gives the same answer every run, whatever the book's length. But it has one blind spot by construction. A character can betray knowledge **without ever naming it** — acting on the secret, or acting conspicuously ignorant of a thing they plainly know — and no name-match can see that, because there is no name to match.

For exactly that case there is an opt-in pass, and only that case:

*The command and its flags*

```
inkhaven knowledge          # the deterministic check
inkhaven knowledge --json   # findings as JSON
inkhaven knowledge --deep   # + the implied-irony pass
    [--max-cost 8000]       #   soft budget for --deep
    [--book-name SLUG]      #   restrict to one book
```

That `SLUG` is a **book's slug** — the short, hyphenated handle Inkhaven derives from a book's title (`The Ninth Lantern` becomes `the-ninth-lantern`). You never have to memorise it. `inkhaven list` prints the project tree with every book's slug in the brackets, and `--book-name` will match a book's **title** just as happily as its slug — so `--book-name "The Ninth Lantern"` works too.

*inkhaven list — the slug is in the brackets*

```
$ inkhaven list
├─ The Ninth Lantern  [book, the-ninth-lantern]
│    …chapters…
├─ Places             [book, places]
├─ Characters         [book, characters]
└─ Facts              [book, facts]
```

Most projects hold a single manuscript and never need the flag; reach for it only to scope the check to one book among several. Pass a slug it does not recognise and Inkhaven simply lists the books it **does** know, each with its slug, so a wrong guess is self-correcting rather than a dead end.

`--deep` hands each scene and a ledger of who-learns-what-when to a model and asks for the **implied** breaks — Sella pouring two cups before Mira has said a word about a second visitor, a character who steps around the dark lantern too carefully for someone who believes it an accident. These land as soft `implied_irony` notices, never hard breaks; the deterministic layer owns the hard calls. The pass is **explicit, cost-capped, and off by default**, on the same rail as the world fact-checker — you pay for it only when you ask, and its cost is previewed against your daily cap before it runs. There is, by deliberate design, no **have the AI judge what everyone knows** mode. That would be guessing, and KEN does not guess.

### DOES IT WORK IN RUSSIAN?

Yes. KEN inherits its reach from the layers beneath it: the topic matcher is Unicode-aware, and the dialogue attribution that decides **who said this line** ships English, Russian, German, French, and Spanish conventions. A secret named in Cyrillic dialogue is caught exactly as one named in English. The tags — `secret:`, `know:`, `reveals:` — are the same in every project language; only the topic between the colon and the end is yours to write.

### WHAT YOU LEARNED

- **KEN** watches one axis nothing else does — **who knows what, and when**. It is SENTINEL's **referenced-before-introduced** move carried into knowledge: a character **acting on a fact before they could learn it**. It **derives** each character's knowledge from structure and never guesses.
- Declare a secret with a `secret:<topic>` tag on the sentence that states it; the words after the colon are the **topic** KEN hunts for in the prose. Tags are not typed into the prose: `Ctrl+B ]` sets and reads back a paragraph's tags, `Ctrl+B }` finds every paragraph carrying a given tag — the same two chords for every tagged feature.
- Grant knowledge two deterministic ways: **presence** (a character in a timeline event's participant list knows it, free) — with a `reveals:<topic>` tag to bind a terse event to the real topic — and **declaration** (`know:<topic>` grants the scene's POV, `know:<topic>@<name>` a named character).
- A use before the earliest grant is a hard break: `premature_knowledge` for an ordinary tracked topic, escalating to `leaked_secret` once the topic is marked `secret:`. A declared reveal that never surfaces again is a soft `dropped_reveal` notice.
- Run it with `inkhaven knowledge` (`--json`, `--book-name`, `--deep` for the opt-in `implied_irony` pass); it **exits non-zero on any break**, so it gates CI. `Ctrl+B Shift+Z` opens the dashboard — `↑↓` scroll, `Enter` jump.
- KEN flags; it never rewrites. Every break is a **decision** — move the line or add a grant — that you resolve in the Editorial Pass, not a correction the tool imposes.

CHAPTER 6

# 6

# Voices and Threads

By now **The Ninth Lantern** has a world, a cast, and a secret. Mira Fenn goes looking for a keeper who is gone; Sella Vale, the harbourmaster, holds the town's official story; Toft the oil-merchant is the wrong suspect everyone reaches for first; Bryn Crane drifts back into town knowing more than he says. On the page, though, a cast is not five people until it **reads** as five people — and a mystery is not a mystery until every promise it makes is one the book remembers to keep. This chapter is about those two jobs: making the voices distinct, and tracking the plot's promises from the moment they are made to the moment they pay off.

Nothing here rewrites a word. The tools in this chapter **read** the draft and report what they find — a pair of characters who blur together, a viewpoint that slips, a

line of speech no one can be sure is spoken, a promise the manuscript has not yet redeemed. Every fix stays yours. They are advisory readers, and the first of them holds the whole cast in mind at once.

## Do they sound like themselves?

The reader we reach for first is **CHORUS** — the voice reader at book scale (the manual's Chapter 18 is its full tour). It profiles each character's dialogue on the same axes the narrator is measured on — sentence rhythm, lexical diversity, hedging, interiority — and then does the thing every novelist is quietly afraid of: it lines the voices up against one another and looks for two that read the same. Run it on the draft:

● ● ● *inkhaven chorus voices — a signature card + the matrix*

```
Character voices — `The Ninth Lantern` [en]

  Mira            confidence ●●●●   Δ from cast mean
    rhythm      ▆▆▆▁   clipped          −0.2
    diversity   ▆▆▆▆   healthy          +0.3
    hedging     ▆▁▁▁   asserts          −0.4
    interiority ▆▆▆▁   high             +0.5

  Distinctiveness
    ⚠ Sella ↔ Toft  read alike (z-dist 0.34 < 0.50)
    ✓ all other pairs distinct

  Aldous          confidence ●○○○   too few lines to judge

  Drift
    ● Bryn  ch 7 drifts from his ch 3 voice
```

Three things in that one screen are worth slowing down for. The headline is the ⚠ line: **Sella and Toft read alike.** On reflection it is exactly the pair you would expect to blur — both are older, transactional, town-official in register, both speak in short declaratives about supply and duty — but you did not **notice** it while writing them a chapter apart. CHORUS z-scores every voice against your own book's spread, so the comparison is relative to **this** manuscript, not to some external notion of "distinct"; a z-distance of `0.34`, under the `0.50` floor, means their fingerprints sit almost on top of each other.

The second thing is Aldous. He is missing from the opening and barely speaks, so he shows a ●○○○ confidence badge and **no** verdict. CHORUS profiles sparse speakers but refuses to flag a voice it cannot measure — it will not tell you Aldous blurs with anyone, because it does not have the lines to know. That restraint is deliberate: a reader that cried wolf on every under-written walk-on would be one you learned to ignore.

The third is Bryn, flagged for **drift** — his chapter-7 voice has wandered from where it started in chapter 3. Drift is always measured against a character's **first** chapter, which turns a vague worry ("does Bryn still sound like Bryn?") into a specific place to look.

> **THE FLAG IS A QUESTION, NOT A VERDICT**
>
> CHORUS measures **consistency** and **distinctiveness**, never quality. "Sella and Toft read alike" is not "one of them is badly written" — it is "a reader may not be able to tell them apart on dialogue alone." If two voices are **meant** to echo — twins, a chorus of identical guards — you tell CHORUS once with `chorus.distinct_ignore_pairs` and it stops raising them. The judgement of whether the echo is a problem stays with you.

The fix for Sella and Toft is authorial and small: give one of them a verbal tic the other lacks. Toft hedges and qualifies (he is, after all, covering for a delivery he cannot account for); Sella never hedges — she **rules**. Sharpen that contrast in a half-dozen lines and their fingerprints separate. You do not need CHORUS to make the change; you need it to have told you the change was owed.

### The Inner Stylist — turning the numbers into judgement

The signature cards are numbers. To hear them as advice, open the **Inner Stylist** — the seventh member of Inkhaven's inner-reader family, the one whose whole job is voice at scale. You reach it through the family hub: `Ctrl+B J` opens the Inner Socrates overview, and `Y` from there opens the Stylist.

● ● ● *Ctrl+B J → Y — the Inner Stylist overview*

```
┌─ Inner Stylist · voice at scale ──────────────────────┐
│ Praise                                                │
│   the cast reads distinct — 9 of 10 pairs separate    │
│ Note                                                  │
│   Bryn's voice drifts plainer after ch 7              │
│ Concern                                               │
│   Sella ↔ Toft read alike — consider sharpening one   │
│                                                       │
├───────────────────────────────────────────────────────┤
│ F synthesise → Output   E engage AI coach → Thoughts  │
│ R report dashboard      Esc close                     │
└───────────────────────────────────────────────────────┘
```

The Stylist reads all of CHORUS's measurements and offers a few grounded observations in the inner family's own register — **Praise**, **Note**, **Concern**, "I notice…", never a rewrite. `F` synthesises them to the Output pane, `R` opens the full voice dashboard, and `E` engages an AI coach into the Thoughts pane for grounded coaching (still never a rewrite). A judgement you have weighed and rejected — say you decide Sella and Toft are **supposed** to sound like two faces of the same officialdom — can be silenced for good with `chorus stylist --suppress <key>`, so it never nags you again.

**FICTION**

In a novel you want the cast to **diverge** — Mira must not sound like Sella. The ⚠ read-alike flag is a warning that two people have collapsed into one voice, and the fix is to sharpen a contrast until the reader can tell them apart in the dark.

**NON-FICTION**

In a multi-author or multi-section non-fiction book the goal **inverts**: you want the chapters to **converge**. Run CHORUS across the sections and a voice that reads alike is good news — it means the seams don't show. Here you hunt the **outlier**: the co-author whose chapter sits apart, the appendix that drifts casual. Same instrument, opposite reading.

## Whose eyes are we behind?

Two viewpoint errors survive line-editing because they are structural, not local — they live in the shape of a scene, not in any one sentence. The first is the **head-hop**: a scene that belongs to one character suddenly showing you the inner life of another. In a Mira scene, `Toft wondered whether…` is a leak — we are not behind Toft's eyes, and for one clause the camera jumped heads.

CHORUS can only catch this if it knows whose scene it is, so you tell it. A scene's viewpoint is declared with a paragraph tag — the same lightweight tag mechanism the rest of Inkhaven uses — placed on the scene's opening paragraph:

● ● ● *Declaring a scene's viewpoint with a pov: tag*

```
pov:Mira         single POV — anyone else's inner life
                 is a leak

pov:first        first person — ANY named character's
                 interiority is a leak

pov:omniscient   deliberately multi-POV — head-hop off
pov:multi        an alias for pov:omniscient
```

**The Ninth Lantern** is a close-third mystery: we ride with Mira and learn only what she learns — which is the engine of the whole book, because the reader must not know what Aldous knew before Mira uncovers it. So every scene of hers carries `pov:Mira`. Where you forget the tag, CHORUS falls back to **inferring** the viewpoint from who is mentioned most, and says so. Now run the discipline scan:

● ● ● *inkhaven chorus scan — POV, head-hop, tense, register*

```
Voice-discipline scan — `The Ninth Lantern` [en]

  POV / head-hop
    ⚠ ch 4 · Mira-POV — "Toft wondered" (interiority leak)
    ● ch 2 · POV inferred (Sella) — no pov: tag

  Tense
    ✓ past-tense throughout — no slips

  Register
    ● ch 7 drifts plainer vs ch 1 (Δ 0.10 > 0.08)
```

The chapter-4 catch is the real prize. In the scene where Mira first suspects Toft, you wrote a line from **inside** Toft's head — "Toft wondered if she could smell the missing oil on him" — a small, natural slip that reads fine in isolation and quietly breaks the book's contract with its reader. CHORUS flags it because the scene is tagged `pov:Mira` and Toft is not Mira. The chapter-2 ● is softer: a Sella scene with no `pov:` tag, so CHORUS inferred her viewpoint and is telling you it guessed. Tag it and the guess becomes a rule.

CHORUS is honest about its own limits here. It has no grammar parser, so it catches interiority attributed to a **named** subject — "Toft wondered" — but not a bare pronoun whose antecedent is someone other than the viewpoint. The tense check, likewise, is a four-language heuristic: it confirms **The Ninth Lantern** holds its past tense throughout, and it would flag a lapse into present — but it covers English, German, French, and Spanish only.

### RUSSIAN IS EXCLUDED FROM THE TENSE CHECK BY DESIGN

If you were writing this book in Russian, `chorus scan` would tell you plainly that it is **not** checking tense — because Russian narrative tense is governed by aspect, and the historical present and perfective/imperfective interleaving are legitimate devices, not slips. A naïve past-to-present flag would be **wrong** there, so CHORUS declines rather than false-flag. Character voice and head-hop still work in Russian; only the tense pillar bows out.

## Who is talking?

Dialogue is the most demanding prose mode there is: a reader must always know who is speaking, an over-decorated attribution verb pulls the eye, and an unbroken run of talking heads floats the scene off its floor. The **DIALOGUE** reader (DIALOG-1) measures speech as a mode of its own — deterministically, in the five languages, with no model call. Run the scan:

*inkhaven dialogue scan — a pre-submission gate*

```
Dialogue scan — `The Ninth Lantern` [en]
──────────────────────────────────────────────────────
  dialogue spans detected   28
  zero attribution           2
  said-bookism density       ch 5 above baseline
  talking-head sequences     1
──────────────────────────────────────────────────────
  [ch.5 · a1f2…] "…she said, he ejaculated" — non-neutral
                 tag verb above the book's own baseline
  [ch.6 · 9c3b…] unattributed speech — no tag or
                 character name within range
──────────────────────────────────────────────────────
```

Two findings, both worth acting on. The chapter-5 flag is a **said-bookism**: in the tense exchange on the quay you reached past `said` for `he ejaculated`, and the density of these ornamental tags in that chapter has climbed above your own book's baseline. DIALOGUE does not ban them — a single `whispered` in the right place earns its keep — it tells you when a chapter has started leaning on them. The chapter-6 flag is **zero-attribution**: a line of speech with no tag and no nearby name, in a spot where the reader cannot be sure whether it is Bryn or Mira talking. Note that DIALOGUE allows an established two-speaker exchange to run untagged for a stretch before it fires — readers track a back-and-forth fine — so this flag means the exchange genuinely lost its footing.

Because zero-attribution is the one dialogue fault a reader **cannot** recover from, `dialogue scan` exits non-zero when it finds one, which lets you wire it into a pre-submission check — `inkhaven dialogue scan --findings zero-attribution || echo "fix attribution first"`.

### The fingerprint view — Ctrl+V Shift+Q

`inkhaven dialogue profile` builds a six-metric signature for each character from the lines confidently attributed to them — terse or expansive, asking or declaring, hedging or asserting:

● ● ● *inkhaven dialogue profile — per-character signatures*

```
Character    Utts  Avg words  MATTR  Quest.  Excl.  Hedge
Mira           41        9.4   0.72    0.29   0.05   0.014
Sella          33       10.1   0.68    0.10   0.18   0.031
Toft           22        9.8   0.66    0.12   0.16   0.028
Bryn           19       13.6   0.71    0.21   0.03   0.037
```

Read across Sella's and Toft's rows and CHORUS's earlier warning turns concrete: their numbers sit almost on top of each other — similar length, similar low question rate, similar exclamatory heat. Mira, terse and questioning, and Bryn, expansive and hedging, stand clearly apart. This is the same story the distinctiveness matrix told, now in the dialogue's own figures.

In the editor, `Ctrl+V Shift+Q` opens the fingerprint view for the **nearest** character — one named in the paragraph you are sitting in, else the most-speaking — drawn as ASCII bars with a compare line for the next speaker. The mnemonic is `Q` for **Quote**. It is built from confidently-attributed lines, so run the review pass (`Ctrl+B Shift+C`) or a `dialogue scan` once to populate it.

## The promises a book makes

A voice is a thing a book **has**; a thread is a promise a book **makes**. **The Ninth Lantern** opens by making a big one — the ninth lantern is cold and its keeper is gone — and the whole mystery is the slow discharge of that promise: suspicion falls the wrong way onto Toft, Mira follows Aldous's trail onto the Long Mole, and the reveal lands (Aldous put the lantern out **himself**, and why). A thread is how you make that promise explicit so the book cannot forget to keep it.

> **TERM** **A plot thread**
>
> A named narrative arc — the cold-lantern mystery, Bryn's family grudge, the town's buried bargain — tracked from its opening hook through its midpoint pivot to its payoff. In Inkhaven a thread is an HJSON-fronted paragraph under the **Threads** system book, carrying a status, a weight, its opening/midpoint/payoff sentences, and links to the characters, places, and artefacts it touches.

You create one from the shell. The lantern mystery is the spine of the book, so it is a `major` thread, and it starts in `setup`:

● ● ● *inkhaven thread add — seeding the spine of the mystery*

```
$ inkhaven thread add "the-cold-lantern" \
      --status setup --weight major
added thread `the-cold-lantern` to Threads (setup · major)
  open Threads/the-cold-lantern in the editor to fill
  opening / midpoint / payoff
```

That seeds a paragraph under the Threads book with a commented HJSON skeleton you fill in the editor. Status runs from `setup` through `develop` to `payoff`, and then to `resolved` or `abandoned`; weight is one of `major`, `subplot`, `runner`, or `bridge`. For the cold-lantern thread you write its arc out in three sentences and mark it `payoff`, because the reveal chapter is the one you are about to draft:

*Threads/the-cold-lantern — the arc, filled in*

```
{
  title:     "The cold lantern"
  status:    "payoff"
  weight:    "major"

  opening:   "The ninth lantern is found dark and its
              keeper gone — an accident, the town assumes."
  midpoint:  "Mira follows Aldous's trail out onto
              the Long Mole and into the fret."
  payoff:    "Aldous put the lantern out himself —
              and what the light was really holding back."

  characters: ["mira", "aldous", "sella"]
  places:     ["long-mole", "ninth-lantern"]
  tension:    9
}
```

A thread on its own is just a note; it earns its keep by being **linked** to the manuscript. As you draft the scenes that raise and develop the mystery, you attach each to the thread with the ordinary paragraph-link chords — `Ctrl+V A` adds an outgoing link, `Ctrl+V I` an incoming one — no new machinery. The links are what let the thread reader see whether a promise is being kept in the actual prose, or only in your intentions. Which is exactly what the doctor checks.

### The doctor — a promise not yet paid off (Ctrl+V Shift+D)

`inkhaven thread doctor` (the chord is `Ctrl+V Shift+D`) reads every thread, tallies how many manuscript paragraphs link to each, and reports the blind spots — the promises the book has made but not yet visibly kept:

● ● ● *inkhaven thread doctor — the blind-spot report*

```
Thread doctor

  threads defined : 3
  avg tension     : 6.3

  status:
    payoff     1
    setup      1
    develop    1

  weight:
    major      1
    runner     1
    subplot    1

Blind spots
  PAYOFF UNFIRED — status `payoff` but no paragraph links:
    · the-cold-lantern
```

There it is: **PAYOFF UNFIRED.** You marked the cold-lantern thread `payoff` — the reveal is the whole point of the book — but not one manuscript paragraph links to it yet, because you have not written the reveal chapter. The doctor is telling you, in the plainest terms, that the book's central promise is **set up but not yet delivered**. That is not a bug in your draft; at this stage it is a to-do list of one, and precisely the one you most need to keep in view. When the reveal is written and its paragraphs linked back to the thread, the flag clears itself.

The doctor names two other blind spots on longer books. **ZERO LINKS** catches a thread whose status has moved past `setup` but which nothing in the manuscript points at — a subplot you developed in your head but not on the page. **DORMANT** catches a `develop` thread with only a link or two project-wide — a promise that is technically alive but has gone quiet. Between them they answer the question a long mystery lives or dies by: **have I paid off everything I set up?** And when you want the whole ledger outside the terminal — for a co-writer, or a submission packet — `inkhaven thread export --format markdown` (or `json`, or `csv`) writes the inventory out.

**FICTION**

In a novel the threads are plot promises — a cold lantern, a buried bargain, a Chekhov's gun on the mantel. **PAYOFF UNFIRED** is the reader's unmet expectation made mechanical: the doctor will not let the reveal you promised slip out of the finished book unpaid.

**NON-FICTION**

In non-fiction the threads are **claims and undertakings** — "we will show that X," "this we return to in Chapter 9." Track each as a thread from where you promise it to where you discharge it, and the doctor's blind-spot report becomes a list of arguments you opened and never closed — the non-fiction equivalent of a dropped subplot.

Between them, CHORUS and the Threads reader answer the two questions that turn a pile of scenes into a book that holds: **do these people sound like themselves**, and **does the book keep the promises it makes?** Neither reader touches your prose. They hand you a sharper contrast to write and a promise to keep — and the writing of both stays where it belongs, with you.

### WHAT YOU LEARNED

- **CHORUS** profiles the cast at book scale. `inkhaven chorus voices` builds the **distinctiveness matrix** and flagged that **Sella and Toft read alike**; sparse speakers like Aldous are profiled but never judged; **drift** is measured against a character's first chapter. The **Inner Stylist** (`Ctrl+B J → Y`) turns the numbers into Praise / Note / Concern.
- Declare a scene's viewpoint with a `pov:Mira` paragraph tag. `inkhaven chorus scan` then catches **head-hops** — "Toft wondered" in a Mira-POV scene — plus tense slips (English/German/French/Spanish; **Russian excluded by design**) and register drift.
- **DIALOGUE** reads speech as its own craft: `inkhaven dialogue scan` flags **said-bookisms** ("he ejaculated") and **zero-attribution**, and exits non-zero as a gate; `inkhaven dialogue profile` and `Ctrl+V Shift+Q` show per-character fingerprints.
- Track a plot promise as a **thread**: `inkhaven thread add "the-cold-lantern" --weight major`, fill its opening/midpoint/payoff under the Threads book, and link scenes to it with `Ctrl+V A` / `Ctrl+V I`. `inkhaven thread doctor` (`Ctrl+V Shift+D`) flagged the reveal as **PAYOFF UNFIRED** — set up but not yet paid off.
- Every reader here is **advisory**: it measures and reports, never edits. The fix — a sharper voice, a tagged viewpoint, a written reveal — always stays yours.

PART IV

# Revision

CHAPTER 7

# 7

# The Read-Through

The draft of **The Ninth Lantern** is done. Not finished — done in the sense that there is now a first chapter and a last one and a whole book in between, cold lantern to the choice on the Mole, with nothing left blank. Every reader so far has read the book **small**: the Inner Editor a paragraph at a time, SENTINEL a chapter break at a time, CHORUS a character at a time. None of them has read it the way the one reader who matters will — cover to cover, in order, once, knowing nothing of the ending until it arrives. That reader is who you become the day the book ships, and you cannot be them, because you know too much. You know Aldous put the lantern out himself. You cannot un-know it and feel the middle drag the way a stranger will.

**LECTOR** is Inkhaven's model of that stranger (the manual's Chapter 18 is its full tour). It reads the finished manuscript forward, once, carrying only what a first reader would carry, and reports two different things you must keep apart in your mind: the **shape** of the read — its structure and its pacing — and the **experience** of the read — its clarity, its attention, whether the stakes are legible and the promises paid. It rewrites nothing. It is a reading instrument, and this chapter is where we finally point it at the whole book.

## The shape of the read

The Shape half measures each chapter's dramatic **intensity** straight from the prose — no tagging, no plan required. It reads the signals a reader feels as rising tension (dialogue density, a stakes-and-conflict lexicon keyed to the project language, sentence-rhythm acceleration), penalises passages that **summarise** instead of **dramatise**, and rewards a chapter that ends on a turn. That gives the **realised** curve — how the book actually rises and falls — which it lays against the **intended** curve of a story framework.

Which framework? **The Ninth Lantern** is a quiet mystery whose engine is not a fight but a **recognition**: the whole book pivots on the moment the cold lantern stops being an accident and becomes a deliberate act. That is the twist-driven shape the East-Asian **kishōtenketsu** structure describes exactly — its energy peaks not at a conflict climax but at the **ten**, the recontextualising turn — and because it is conflict-optional, a tension-light book is not scored as though it had failed to be a thriller. So we set the framework by hand rather than let the default `genre` guess pick the Seven-Point:

● ● ● *inkhaven.hjson — pinning the intended shape*

```
lector: {
  framework: "kishotenketsu"   // ki · shō · ten · ketsu
}
```

The `lector:` block lives in your project's `inkhaven.hjson` (open it by hand, or with `Ctrl+B 0`). `framework` takes any of the six shapes Inkhaven knows — `three_act`, `save_the_cat`, `story_circle`, `hero_journey`, `seven_point`, and `kishotenketsu`. Leave it unset and the read-through guesses one from the

book's genre; name it by hand, as we do here, when you already know the shape you are writing to.

Now run the read-through. It prints the whole read in one shot — the curve, the per-chapter beat, and the ranked findings:

● ● ● *inkhaven readthrough — the measured curve vs kishōtenketsu*

```
Read-through — 9 chapter(s) · Kishōtenketsu
  measured   ▂▁▁▂▅▇▃▅
  expected   ▁▂▃▅▆▇▅▃

  ch  1  ▂ ▶   The Cold Lantern
  ch  3  ▂ ◉   What Toft Owed
  ch  5  ▁ ◉   Old Debts
  ch  6  ▄ ▶   Onto the Mole
  ch  7  ▇ ▶   What the Light Held
  ch  9  ▂ ◉   Relit

⚠ [shape_sag] the Kishōtenketsu shape wants rising tension
              around ch. 5 (~55%) but the prose reads flat (~14%).

4 reader finding(s): 1 concern, 3 notice(s).
```

Read the two sparklines together and the book's problem is drawn in eight characters. The **expected** line climbs steadily through the **shō** — the development movement — toward the **ten** at chapter 7, where Mira learns what the light was really holding back. The **measured** line does the opposite in the middle: it **dips** at chapters 4 and 5, exactly where the book most needs to be tightening. That is the saggy middle every mystery is prone to — the stretch after the wrong suspect (Toft) is set aside and before the trail onto the Mole picks up, where Mira searches and re-searches and the prose is **reporting** her search rather than dramatising it. LECTOR names it `shape_sag` and gives you the coordinates: chapter 5, about 55% of the way in, where the framework wants a rise and the prose reads flat.

> **THE FLAG MEASURES THE READ, IT DOES NOT PRESCRIBE THE FIX**
>
> `shape_sag` does not say "add a fight in chapter 5." It says a first reader's attention will slacken there against the shape you intended. The remedy is authorial and it is yours: cut the second search scene, or give the middle its own small turn — Mira finds the circled date, say, and misreads it — so the **shō** actually develops instead of marking time. LECTOR told you **where**; the page is still your job.

Alongside the curve, each chapter carries a **beat** on the scene ⇄ sequel axis. A **scene** is the forward, external unit — goal, conflict, disaster — and shows `▶`; a **sequel** is the reflective, internal one — reaction, dilemma, decision — and shows `◉`; a chapter that is neither reads `·`. The value is in the **rhythm**, not any single chapter: a long run of pure scene reads breathless with no room to feel the cost, and a long run of pure sequel **sags**. Look back at the beat column and the sag has a second signature — chapters 3 and 5 are both `◉`, reflective back to back, with the search chapter between them carrying almost no forward motion. The intensity dip and the sequel run are the same weakness seen two ways.

## The experience of the read

The second half is the reader's **experience**, and its defining discipline is that it is **forward-only**. LECTOR walks the book from chapter one carrying the state a first reader would carry — which characters and places have been met, which threads hang open, how the energy has been running — and every finding uses only the chapters read **so far**. A payoff in chapter 8 can never reach back and cancel a confusion the reader felt in chapter 3, because the reader in chapter 3 had not read chapter 8. That single rule is what makes this a **reader** and not an **analyst**, and it is why its findings are the ones worth trusting. Five come out of the walk, all deterministic and free:

*The five forward-walk findings*

```
confusion       an entity used before it is introduced —
                "who is this again?"

info_dump       too many new names to meet in one chapter

attention_dip   a flat, eventless chapter where attention
                drifts

put_down_risk   a RUN of flat chapters — a likely place the
                reader sets the book down

unpaid_setup    a detail planted early and never paid off
```

Two of them fired on **The Ninth Lantern**, and both are the kind of fault you cannot see from inside your own book:

*inkhaven readthrough — the ranked reader findings*

```
Reader findings — `The Ninth Lantern`

  ⊗ put_down_risk   ch 4–5 run flat — a likely put-down point
  ⚠ shape_sag       ch 5 wants a rise, the prose reads flat
  ⚠ confusion       "Bryn" used in ch 3, first met in ch 5
  ⚠ unpaid_setup    "the circled date" (ch 2) — never paid off
```

The `confusion` catch is the one that stings, because it is invisible to the author by construction. In chapter 3, writing Mira alone in Aldous's cottage, you had her think **"not since Bryn left"** — a line that means everything to you, because you know Bryn is Aldous's estranged nephew and you know he is about to walk back into the story in chapter 5. But the **first reader** has never heard the name. To them "Bryn" is a stranger dropped into a sentence as though they should recognise him — **who is this again?** — two chapters before he is introduced. LECTOR flags it because the walk met the token "Bryn" in chapter 3 and did not meet the **character** Bryn until chapter 5, and it reports the gap. The fix is one clause: make chapter 3's mention introduce him — **"not since Aldous's nephew Bryn left"** — or move it later. Either way, a slip you could not have felt is now a line you can see. This detector shares the Unicode-aware mention matcher SENTINEL uses, so it works in every script Inkhaven supports, not English alone.

The `unpaid_setup` catch is the mystery writer's other recurring sin: a detail raised with weight and then forgotten. In chapter 2, searching Aldous's post, Mira notices his tide-chart pinned over the cot with **one date circled in ink**. It reads like a clue — you meant it as atmosphere — and the reader files it as a promise. But the book never returns to it: the circled date is never explained, never paid. LECTOR walked the whole book forward, saw the setup opened and never closed, and raised it. Now you have a clean choice, and it is a **good** choice to have: pay it off (the date is the anniversary of the town's buried bargain — the thing the lanterns hold back) and the detail becomes a thread; or cut it, and the reader stops waiting for a bill that never comes. Left alone, it is exactly the loose end a review will name.

> **FICTION**
>
> In a novel these are plot faults: a name used before the reader can place it, a Chekhov's gun that never fires. The forward walk is the beta reader who does not yet know your ending and cannot fill your gaps with what you know.

> **NON-FICTION**
>
> In non-fiction the same two flags serve an **argument's** flow. `confusion` is a term used before it is defined — a piece of jargon the reader meets three pages before the definition. `unpaid_setup` is a claim you promise to support ("we return to this in Chapter 9") and never do. Same instrument, reading the logic of an exposition instead of the shape of a plot.

## The synthetic first-read — the one explicit cost

Some things the deterministic walk simply cannot judge: whether a passage is **genuinely** confusing to a human, whether the stakes are **legible** on the page, whether engagement is **really** flagging where the intensity numbers merely dip. For those, a model helps. LECTOR's one LLM feature — the **synthetic first-read** — reacts to each chapter as a first reader who does not know the ending. It is forward-only by construction: each call sees only a recap of what has been read plus the current chapter, never the whole book at once.

It is never automatic. You ask for it — `inkhaven readthrough --deep`, or the `k` key in the dashboard — and before each chapter the estimated cost is previewed

against your daily cap. Per Inkhaven's permissive principle the cost **informs**; it never blocks. On our saggy middle, the synthetic reader says out loud what the `shape_sag` number could only imply:

*inkhaven readthrough --deep — the synthetic first-read*

```
inkhaven readthrough --deep [--max-cost 8000] [--force]

ch  5  synthetic first-read  · est. ~1,150 tokens (cap ok)
→ I keep waiting for something to happen. Mira searches
  the cottage a second time and I already know the room.
  Who is Bryn? — the name lands like I've missed a page.

ch  7  synthetic first-read  · est. ~1,290 tokens (cap ok)
→ Oh — he put it out himself. That reframes the whole
  search; I want to go back and reread chapter 1.
```

That chapter-7 reaction is the sound of a working **ten**: the reader recontextual- ises everything behind them. The chapter-5 reaction is the sag felt from the inside — boredom and the same stray "who is Bryn?" the deterministic walk caught, now voiced. The two passes agree, which is the point: the free walk finds the **where**, and the paid read confirms the **feel**, and you pay for the model only when you choose to. Its findings arrive tagged `source: reader` and land beside the deterministic ones, always marked so you know which came from a model.

## The dashboard — Ctrl+B Shift+A

The command line is the batch view; inside the editor the read-through has its own dashboard. `Ctrl+B Shift+A` opens a scrollable modal with the curve, the beats, and the ranked findings together:

*Ctrl+B Shift+A — the read-through dashboard*

```
┌─ Read-through · Kishōtenketsu · 9 ch ──────────────┐
│  measured   ▂▁▂▃▆▇                                 │
│  expected   ▂▃▄▅▆▇                                 │
│                                                    │
│ ⊗ put_down_risk   ch 4–5 run flat — likely put-down │
│ ⚠ shape_sag       ch 5 wants a rise, reads flat     │
│ ⚠ confusion       "Bryn" used ch 3, met ch 5        │
│ ⚠ unpaid_setup    "the circled date" (ch 2) unpaid  │
│                                                    │
├────────────────────────────────────────────────────┤
│ ↑↓ scroll   Enter jump to chapter   k synthetic-read│
│ Esc close                                          │
└────────────────────────────────────────────────────┘
```

Arrow keys scroll it; `Enter` jumps straight to the flagged chapter in the editor — select the `confusion` line and you land in chapter 3 on the stray "Bryn", ready to fix it; `k` runs the cost-capped synthetic first-read, whose reactions post to the Output pane's `readthrough` category; `Esc` closes. The deterministic findings also ride the unified review pass, `Ctrl+B Shift+C`, which adds a `read-through` line — each finding anchored to its chapter — alongside everything else the book noticed about itself.

*The LECTOR surface at a glance*

```
inkhaven readthrough            the full report
inkhaven readthrough --deep     + synthetic first-read
inkhaven readthrough --json     structured output

Ctrl+B Shift+A    the dashboard (k = synthetic read)
Ctrl+B Shift+C    read-through line in the review pass

ink.readthrough.report   the ranked findings
ink.readthrough.curve    per-chapter measured/expected
ink.readthrough.check    counts by kind + severity
```

The three `ink.readthrough.*` Bund words expose only the **deterministic** read — the synthetic first-read is deliberately not scriptable, because a cost-bearing model call has no business firing silently from a hook. That is the same line Inkhaven draws everywhere: the free, deterministic measurement is always available to your scripts; the model call stays an explicit, deliberate act.

> **WHAT THE READ-THROUGH IS NOT**
>
> It is not a rewriter — it reports the read and hands every fix to the Editorial Pass (Chapter 8), where a finding becomes an action under the confirmed-diff, snapshot-first contract. It is not a per-paragraph reader — it is whole-book, forward, once. And it is not an oracle of taste: `shape_sag`, `confusion`, `unpaid_setup` are legible, grounded signals — a flat stretch, an unmet name, a loose end — never a verdict on whether **The Ninth Lantern** is **good**. That verdict is still yours to earn.

The read-through is the first time the book has been looked at **whole**, the way a reader will look at it, and it did what only that vantage can: it found the middle that drags, the name that arrives too early, the clue that never pays. None of these was visible from inside a paragraph, and none of them is a change LECTOR will make for you. What it hands you is a short, honest list of the places a stranger will stumble — and, in the next chapter, a disciplined way to act on it.

### WHAT YOU LEARNED

- **LECTOR** (`inkhaven readthrough`) reads the finished book forward, once, as a first reader, and reports two things: the **shape** of the read and the **experience** of it. On **The Ninth Lantern** we pinned the `lector` framework to `kishotenketsu` because the book peaks on a recognition (the **ten**), not a fight.
- The **Shape** half measured the intensity curve from the prose and laid it against the framework, flagging `shape_sag` at chapter 5 — the search-heavy middle that dips where the shape wants a rise — reinforced by a back-to-back ◉ sequel run on the scene ⇄ sequel axis.
- The **Audience** half walks **forward-only**: it flagged a `confusion` ("Bryn" used in ch 3, two chapters before he is introduced) and an `unpaid_setup` (the circled date on Aldous's tide-chart, planted in ch 2 and never paid) — faults invisible from inside the draft.
- The **synthetic first-read** (`inkhaven readthrough --deep`, or `k` in the dashboard) is the one explicit, cost-capped LLM pass — a chapter-by-chapter reaction that voiced the same sag and confusion the free walk found. Cost informs, never blocks; findings are tagged `source: reader`.
- `Ctrl+B Shift+A` opens the dashboard (curve · beats · findings; `Enter` jumps to a chapter, `k` runs the synthetic read); `Ctrl+B Shift+C` folds the read-through into the review pass. Everything is **advisory** — the fix is handed to the Editorial Pass, never made for you. The non-fiction reading is the same: `confusion` is a term used before it is defined, `unpaid_setup` a claim promised and never supported.

CHAPTER 8

# 8

# The Editorial Pass

Every reader in this part of the book has now had its say about **The Ninth Lantern**, and each said its piece on its own surface. KEN, in Chapter 5, caught Sella naming the reveal a chapter before she could know it. CHORUS, in Chapter 6, found that Sella and Toft read alike. LECTOR, in Chapter 7, felt the middle sag and marked a place a first reader might lose the thread. And back in Chapter 4 the fact-checker noticed the lanterns burning **three hundred years** in one place and **two hundred** in another. Four true findings — and four different screens, four different commands, four different mental notes to hold while you sit down to actually fix the book.

That scattering is the problem this chapter solves. A diagnosis is not a revision, and a dozen diagnoses spread across a dozen surfaces is not a to-do list — it is a scavenger hunt. **REDLINE** ends the hunt. It gathers every reader's findings into one ranked worklist and, for each one, hands you the honest kind of help — a rewrite you confirm, a decision it asks you to make, or a brief it can only advise. Nothing new is computed; REDLINE reads what the readers already found. And — the promise the whole chapter turns on — it never changes a word of your prose without showing you the exact diff and taking a snapshot first. (The manual's Chapter 19 is REDLINE's full reference; this is the pass run on our own book.)

## One worklist, every reader

In the editor, `Ctrl+V Shift+R` opens the **Editorial Pass** — the worklist as a cockpit you can walk. It sits under the view prefix (`Ctrl+V` reaches a tool rather than reshaping the book), and the mnemonic is simply **R** for **Revision**. The pass opens instantly, because it is reading findings the readers have already produced, not running them again. Here is **The Ninth Lantern**'s, the morning after the read-through:

● ● ● *Ctrl+V Shift+R — one worklist, each row tagged how to act*

```
┌─ Editorial Pass · 6 findings (2 err · 2 warn · 2 info) ──────┐
│  ✗ ⇄ leaked_secret  ch. 5 Sella names the reveal before ch. 6 │
│  ✗ ⇄ fact           ch. 4 "200 years" but ch. 1 says 300      │
│  ⚠ ✎ editor         ch. 4 "Mira felt afraid" — line is inert  │
│  ⚠ ✉ distinctiveness  —   Sella and Toft read alike           │
│  · ✉ shape_sag      ch. 3 the middle sags — wants a rise      │
│  · ⇄ confusion      ch. 3 a reader may lose who suspects whom │
├──────────────────────────────────────────────────────────────┤
│ ↑↓ move · [ ] filter · ⏎ jump · f act · F fix-all ·          │
│ s skip · d defer · D clear · Esc close                       │
└──────────────────────────────────────────────────────────────┘
```

Six findings from five different readers, errors first, each carrying one thing no bare to-do list has: a glyph in the second column that tells you **what acting on it will do** before you do it. That glyph is the whole idea behind REDLINE.

> **TERM** **Response kind**
>
> Every finding carries a **response kind** — the honest form of help for that particular problem, shown as a glyph. 🖉 **Rewrite** is a diff-reviewed local prose fix; ⇄ **Decision** is a choice only you can make, after which REDLINE reconciles the prose to your answer; ✉ **Brief** is written advice for a structural problem no single paragraph can solve. Only one of the three — Rewrite — ever touches your words, and even then only through a diff you accept.

The kind is derived from what sort of problem the finding **is**, not guessed. A repeated word or a flat line has an honest single-paragraph fix, so it is a Rewrite. A fact stated two ways, or a secret spoken too early, cannot be adjudicated by a machine — it needs **you** to say which way is true — so it is a Decision. A whole act that sags cannot be honestly patched by rewriting one paragraph at all, so it is a Brief. Read down our worklist and the glyphs sort the morning's work for you: two Decisions to rule on, one line to rewrite, one voice-note and one structural sag to take as advice, and one more Decision about the thread the reader might lose.

Moving through it is quick. `↑ ↓` move the cursor and expand the selected finding's full message in place; `[` and `]` cycle a category filter, so you can walk all the Decisions, then all the Rewrites, one class at a time; `⏎` jumps the editor straight to the paragraph so you can read the finding in its context. The work itself happens on three keys — `f` acts on the selected finding, `F` walks **only** the 🖉 Rewrites one diff at a time, and `s` / `d` skip for the session or defer until the prose changes. Let us walk ours.

## Walking the list

### 🖉 Accepting a rewrite — the flat line

Start with the cheapest win: the `editor` finding at Chapter 4. When Mira first steps onto the Long Mole you wrote **"Mira felt afraid as she stepped onto the Mole"** — and the Inner Editor, reading that paragraph, noticed the line goes inert: it **names** the feeling instead of letting the reader feel it, and the fret that should be closing round her never touches her at all. Press `f`, and this is the marquee case — the Inner Editor's own observation is handed to the model **as the instruction**, so the fix answers that note and not some generic "make it better" recipe.

● ● ● *f on a ✎ Rewrite — the diff you accept, note-driven*

```
┌─ AI diff · ✎ editor · ch. 4 ──────────────────────────────┐
│ the note handed to the model:                             │
│   "names the feeling instead of showing it — the verb is a │
│    bare copula and the fret never touches her"            │
├──────────────────────────────────────────────────────────┤
│ - Mira felt afraid as she stepped onto the Mole.          │
│ + The Mole narrowed to a grey thread ahead, and the cold  │
│ + fret closed over Mira's hands as she stepped out onto it. │
├──────────────────────────────────────────────────────────┤
│ a / e / ⏎  accept    r  reject    (snapshot on accept · F6) │
└──────────────────────────────────────────────────────────┘
```

Nothing has changed yet. The model's rewrite has streamed into the AI pane and a diff-review modal is showing you the exact before-and-after — no more, no less. Now you choose. `a`, `e`, and ⏎ all accept it as written; `r` rejects and the paragraph stays exactly as it was. Reject costs nothing — the flat line simply remains, and you can come back to it or fix it by hand. Accept, and **before** the new prose lands, your old sentence is snapshotted with a labelled entry, so the version you had is one `F6` away for as long as the project lives. That order — snapshot first, then replace — is not an incidental nicety; it is the safety contract, and the next section is about why it holds without exception.

Because this is a plain single-line Rewrite, you could also have batched it: `F` walks every ✎ Rewrite in the list in turn, each one its own diff to accept or reject, `Esc` stopping the run wherever you are. On this list `F` would offer you exactly one diff — the `editor` line — because it is the only Rewrite present. The Decisions and the Briefs are never batched; they need you.

### ⇄ Making a decision — which fact is true

The two red rows at the top are Decisions, and they are red because they are the findings a reader will feel first. Take the `fact` one. The fact-checker noticed the manuscript states the lanterns' vigil two ways: **three hundred years** in the opening of Chapter 1, **two hundred** in Sella's mouth on the quay in Chapter 4. The machine cannot know which you meant — both are grammatical, both are clean prose — so it does not touch either. It asks.

● ● ● *f on a ⇄ Decision — you rule, REDLINE reconciles*

```
┌─ ⇄ Decision · fact · ch. 1 ↔ ch. 4 ──────────────────────────┐
│ The manuscript states this two ways. Which is true?          │
│                                                              │
│   1  three hundred years    ch. 1 · the opening              │
│   2  two hundred years      ch. 4 · Sella on the quay        │
│                                                              │
│ ↑↓ choose · ⏎ confirm — REDLINE reconciles the other to your │
│ answer as a diff you accept + a snapshot · Esc leave it open │
└──────────────────────────────────────────────────────────────┘
```

The bible says three hundred — the lanterns have burned three centuries — so you choose 1. REDLINE does not simply record your ruling; it carries it into the prose, rewriting the Chapter 4 line so Sella says **three hundred**, and it does so through the **same** confirmed-diff-plus-snapshot path the line-rewrite took. A Decision, once decided, becomes a reconcile that you still see as a diff and still accept before it lands. You made the judgement the machine could not; the machine did the retyping the judgement implied, and left the receipt.

The leaked-secret Decision above it works the same way, though its resolution is usually not a rewrite at all: KEN caught Sella naming the reveal in Chapter 5, before she learns it in Chapter 6, and the honest fixes — move the line past her Chapter 6 grant, or grant her the knowledge earlier if she truly should have had it — are structural choices you make in the manuscript, exactly as Chapter 5 described. REDLINE's job was to put that break in the same queue as the flat line, so it is not forgotten, and to mark it a Decision so no tool ever "resolves" a secret on your behalf.

### ✉ Reading a brief — the saggy middle

The last kind is the one an automated editor does the most damage pretending it can fix. LECTOR's `shape_sag` says the middle of the book — Chapters 3 and 4 — holds the same suspicion of Toft without deepening it, so the read goes slack before the Mole. There is no paragraph to rewrite here; the problem lives in the shape, across two chapters. So pressing `f` on it does not open a diff. It writes a developmental **brief** to the Thoughts pane, and stops.

● ● ● *f on a ✉ Brief — advice to the Thoughts pane, no rewrite*

```
◆ Revision brief · ✉ shape_sag · ch. 3–4

The middle sags: the suspicion of Toft is raised, then held
at the same pitch for two chapters, so the read goes slack
before the turn onto the Mole. Three ways to give it a rise:

  · Give ch. 3 a smaller reversal of its own — a detail that
    half-clears Toft — so the suspicion has somewhere to move
    rather than simply persisting.
  · Bring one Mole thread forward: let Mira find the trail-
    head early, so the middle pulls toward the reveal instead
    of marking time until it.
  · Merge the two quay scenes; the second restates the first
    without raising the stake, and the join would tighten both.

(advice only — nothing was written to your prose)
```

That is the whole of what a Brief does: name the problem in a line, give two or three concrete, craft-grounded suggestions tied to **this** sag, and stop. It advises; it never rewrites. The fix — cutting a scene, planting a reversal — is work only you can do, and REDLINE does not pretend a paragraph-sized patch could stand in for it. The brief lands where your other thinking lands, in the Thoughts pane, for you to act on when you next open the middle.

**FICTION**

The three kinds map onto the three problems every draft has: a **line** that reads flat (✎ rewrite it), a **fact** the book states two ways (⇄ decide which is true), and a **stretch** that sags across chapters (✉ take the brief). Our worklist has one of each, plus the mystery-specific leaked secret.

**NON-FICTION**

An argument has the same three. A sentence that clears its throat instead of making its point is a ✎ Rewrite. A claim one of your own sources contradicts is a ⇄ Decision — you rule which the book will stand on. A section that drags before it earns its conclusion is a ✉ Brief. Same worklist, same three kinds of help.

## The safety contract

Everything above rests on one guarantee, and it is worth stating flatly: **REDLINE will not edit your prose on its own.** This is not a setting or a policy layered on top — it is the shape of the code. Every prose change REDLINE can make — a ✎ fix, a ⇄ decision's reconcile, one step of the `F` batch — flows through exactly one path, and there is no other.

● ● ● *Every REDLINE prose change takes the same three steps*

```
the model's rewrite streams into the AI pane
                  |
                  ▼
a diff-review modal shows the exact change
     accept (a / e / ⏎)   or   reject (r)
                  |  accept
                  ▼
your old prose is SNAPSHOTTED first, labelled,
and only THEN replaced
                  |
                  ▼
     recover it any time with  F6
```

Read the three steps as promises. You always see the exact diff — the change is shown, never applied blind. Nothing is written until you accept — reject, and the paragraph is untouched. And on accept your old prose is snapshotted **before** the new text lands, so the version you had survives one keystroke away. There is no code path in REDLINE that writes prose without all three.

### THE F BATCH IS REWRITE-ONLY BY CONSTRUCTION

When you press `F` you are never one keystroke from an unconfirmed edit. The batch queue can hold **nothing but** ✎ Rewrites: a Decision or a Brief has no fix recipe to run in the first place, so it cannot enter the queue — the **type** of the finding forbids it, and a guard test locks the invariant. The batch is a convenience for the mechanical fixes, never an autopilot for the ones that need a human.

## The editorial letter — inkhaven revise

The Editorial Pass is the row-by-row cockpit. Sometimes, though, you want the opposite view first — the **overview** a good editor opens a revision with, before touching a single line. That is `inkhaven revise`. It runs the very same worklist, then synthesises the whole of it into one developmental letter.

● ● ● *inkhaven revise — the letter over the whole worklist*

```
$ inkhaven revise
revise: synthesising the editorial letter over 6 finding(s)…

Dear author,

The Ninth Lantern works because it guards one secret, so the
first thing to mend is the one place it slips: in Chapter 5
Sella already knows what she should not learn until Chapter 6.
Fix that before anything else — it is the whole difference
between a mystery and a misprint.

Continuity & knowledge
  That leaked reveal is the only hard break. A smaller snag
  sits beside it: the lanterns burn "three hundred years" in
  Chapter 1 and "two hundred" in Chapter 4 — one ruling settles it.

Structure & pacing
  The middle (ch. 3–4) holds its suspicion without deepening
  it; give it a reversal so it pulls toward the Mole.

Voice & character
  Sella and Toft still read alike — sharpen one and the quay
  scene stops sounding like one official talking to a mirror.

Line & prose
  A few lines name a feeling instead of earning it ("Mira felt
  afraid"); those are quick, local fixes.

Fix the leak, make the factual ruling, then walk the rest in
the Editorial Pass.
```

The letter opens with the big picture — the one or two things a reader will feel first — then groups the rest by theme, most important first, each with a brief **why it matters** and **what to do**. It advises; it never rewrites — the writing stays in the Editorial Pass's confirmed-diff loop. It is written in the manuscript's language,

and each finding keeps the reader it came from, so the perennial **"does it work in Russian?"** answers itself per detector rather than as one blanket promise. When you want the worklist for a script instead of a person, the same command has a machine face:

● ● ● *The scriptable faces of the same worklist*

```
inkhaven revise                 # the editorial letter, whole
                                # project
inkhaven revise --book-name X   # restrict to one book (slug or
                                # title)
inkhaven revise --json          # the findings as JSON: category,
                                # severity (high/med/low),
                                # response, location, message
```

The `--json` form emits the ranked findings — the AI letter and every prose rewrite deliberately left out — ready for a hook or a report; and the read-only `ink.revise.check` Bund word returns a small tally — findings counted by severity, plus a `clean` flag that is `true` when nothing high-severity remains — a ready-made **is this draft ready?** gate for a script. Advice and prose changes stay author-initiated; only the deterministic findings are scriptable.

That is the pass. You arrived with four readers' findings on four screens and left with a flat line rewritten, a fact ruled on, a brief in hand for the middle, and a secret marked for moving — every prose change a diff you accepted, every old version a keystroke away. What you have not yet is proof that any of it **helped**. That is the next chapter's question.

### WHAT YOU LEARNED

- **REDLINE** gathers every reader's findings — KEN's leaked secret, the fact-checker's contradiction, CHORUS's read-alike voices, LECTOR's sag, the Inner Editor's flat line — into **one ranked worklist**, errors first. The Editorial Pass (`Ctrl+V Shift+R`), `inkhaven revise`, and CHRONICLE all read the same list.
- Each finding carries a **response kind**, derived from the problem not guessed: ✎ **Rewrite** (a diff-reviewed local fix), ⇄ **Decision** (you rule which way is true, REDLINE reconciles the prose to your answer), or ✉ **Brief** (advice for a structural problem, written to the Thoughts pane, never a rewrite). Only Rewrite touches prose.
- Walk it with `f` (act on one), `F` (batch **only** the ✎ Rewrites, one diff each), `[ ]` (filter), ⏎ (jump), `s` / `d` (skip / defer). A Decision's reconcile goes through the same confirmed-diff path a Rewrite does.
- The **safety contract**: every prose change streams a diff you see, is written only on your accept (`a` / `e` / ⏎; `r` rejects), and **snapshots your old prose first** — one `F6` from recovery. There is no other path, and the `F` batch is Rewrite-only by construction.
- `inkhaven revise` synthesises the same worklist into one **editorial letter** — big picture, then grouped by theme — in the manuscript's language; `--json` and `ink.revise.check` expose the findings read-only for a script. It advises; it never rewrites.

CHAPTER 9

# 9

# Did It Get Better?

In the last chapter you spent an afternoon in the Editorial Pass. You reconciled the leaked secret — moved Sella's give-away line so she no longer knows a chapter early. You took the read-through's brief on the sag in the middle and tightened the scenes where the mystery goes slack. You cleared a handful of echoes and a filter word or two. It **feels** better. But feeling is not knowing, and every reviser has had the afternoon that felt productive and left the book no better — a dozen findings closed, a dozen new ones opened three chapters downstream, the net gain a rounding error. So the honest question, the one no reader you have met so far can answer, is the plainest one there is: **did the book actually get better, or did you just move the problems around?**

This chapter is about the one intelligence built to answer it — **CHRONICLE**, the draft historian (the manual's Chapter 19 gives its full tour). It does exactly one thing, and it does nothing else: it **measures**. There is no prose-write path anywhere in it. It cannot rewrite a line, and it will not try. It remembers what your book measured at a milestone, compares that to the book in front of it now, and tells you — in the book's own numbers — whether the revision was a revision.

## The milestone you already had

CHRONICLE can only answer the question because, one keystroke before the revision, you gave it something to measure against. Before the first `✎ Rewrite` of the last chapter — while the draft still held every fault the readers had found — you stamped a **milestone**:

*inkhaven chronicle mark — freezing the draft's verdict*

```
$ inkhaven chronicle mark "first-draft"
✓ marked "first-draft" — 12 finding(s) (1 error · 4 warn · 7 info)
```

That one command runs the same unified worklist REDLINE walks — the shared `collect` behind the whole of Part IV — and freezes the result: the total count, the tallies by severity and category, and, the part that makes everything later possible, the **fingerprint of every single finding**. It is a photograph of the draft's diagnostic state, taken at the moment you decided this draft was worth remembering.

> **TERM** **A milestone**
>
> One named, timestamped capture of the draft's whole diagnostic state — the counts by severity and category **plus** the fingerprint of every finding. Milestones are deliberate: you stamp one with `chronicle mark <label>` before a big pass, before a reader sees it, at the turn of a version. They are the fixed points every trend and diff measures between, and they live in the project's own `chronicle.db`, beside your prose but never mixed into it.

Milestones are never taken behind your back. CHRONICLE is not a daemon and not a git tool — it will not stamp a draft because you saved, and it will not invent

a tag. A draft is a decision, so a milestone is a keystroke. You can list the ones you have, newest first:

● ● ● *inkhaven chronicle list — the marks you have stamped*

```
$ inkhaven chronicle list
2026-08-04  first-draft        12 finding(s)  1✗ 4⚠ 7·
```

One mark so far. That single row is the baseline the rest of this chapter measures against — and the whole reason the afternoon of revision can now be graded rather than merely felt.

## The trend — every count fewer is better

Now do the revision — you already did, in the last chapter — and ask the question. Run `chronicle` with no arguments. Bare, it captures the **live** state of the book right now and diffs it against your most recent mark: the "did it get better since I last looked?" view.

● ● ● *inkhaven chronicle — the live book against your last mark*

```
$ inkhaven chronicle
Chronicle — since "first-draft" (2026-08-04) → now

  findings          12 →  11   ▼
  errors             1 →   0   ▼  cleared
  warnings           4 →   5   ▲
  infos              7 →   6   ▼

  by category:
    echo               0 →   1   ▲  NEW
    leaked_secret      1 →   0   ▼  cleared
    shape_sag          1 →   0   ▼  cleared

  ✓ 2 cleared    ▲ 1 introduced    · 10 unchanged

  introduced (new since the last mark):
    ⚠ echo         ch. 4      "fret" repeats four times — de-echo
```

Read the top block first, because its polarity is the whole point. Every number CHRONICLE trends is a count of findings the readers **raised** — which means every one is **fewer-is-better**. A falling count is an improvement, a rising count a regression, and CHRONICLE scores the direction for you so you never have to work out whether a number moving is welcome. The arrow carries it at a glance: `▼` a count that fell (good), `▲` one that rose (a regression), `·` one that held. A category that dropped to nothing is tagged `cleared`; one that appeared from nothing is tagged `NEW`. The one error you had — the leaked secret — is gone, so `errors` reads `1 → 0 ▼ cleared`. That is the headline you came for.

But look at `warnings`: `4 → 5 ▲`. It went the wrong way. You **added** a warning somewhere while you were fixing everything else, and CHRONICLE will not let that hide behind the good news above it. The regressions sort to the top of the category list precisely so the worst thing you did is the first thing you read — here, `echo 0 → 1 ▲ NEW`, sitting above the two categories you cleared.

> **DIRECTION IS SCORED, NOT LEFT TO YOU**
>
> This is the difference between a dashboard and a diff. A pile of raw counts would leave you squinting at whether `4 → 5` is progress. CHRONICLE knows that every finding is a fault, so it knows `4 → 5` is a regression and says so, in the arrow and in the ordering. You are never asked to remember which direction is good; the tool remembers for you, and puts the bad news on top.

## Cleared versus introduced — the signature

The counts tell you **how much** moved. The line below them tells you **exactly what** — and it is the move that makes CHRONICLE worth having.

● ● ● *The signature line — a set difference over fingerprints*

```
  ✓ 2 cleared    ▲ 1 introduced    · 10 unchanged
```

Because every finding froze with a stable fingerprint at the mark, CHRONICLE can do a plain set difference between the old finding set and the live one, and sort the result into three piles. Nothing here is estimated; it is arithmetic on identities.

| | |
|---|---|
| ✓ **cleared** | There at the mark, gone now — the findings your revision actually resolved. The receipt that the work landed. Here: the leaked_secret and the shape_sag. |
| ▲ **introduced** | New since the mark — the findings your edits, or the ripple around them, created. The early warning on collateral damage. Here: one echo, in ch. 4. |
| · **unchanged** | Present in both — still standing, still waiting for you. Here the ten you never touched, including Bryn's dropped_reveal from the KEN chapter. |

This is what closes the loop the Editorial Pass opened. The **cleared** list is the proof of the work you meant to do: the leaked secret you reconciled and the sag you tightened are named, by category, gone. You did not imagine the improvement — here are its two receipts.

The **introduced** list is the thing no amount of careful rewriting can see from inside a single paragraph. When you tightened the slack scene in Chapter 4 to lift the sag, one of those hurried rewrites reached for the word `fret` four times in two sentences — a fresh echo, in the very paragraph you were fixing. You could not have caught it by re-reading the fix, because it reads fine in isolation; the sentence you wrote to solve one problem quietly made another. The introduced list is **itemised**, not merely counted, so it is a to-do and not a worry: severity, category, the chapter, and the head of the finding, ready to act on.

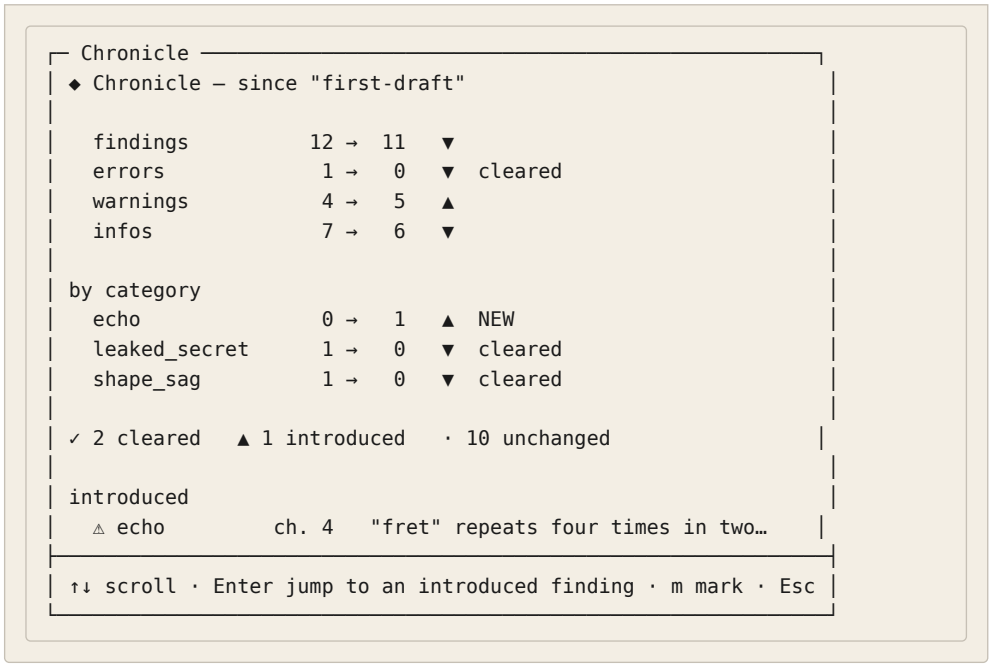
> THE HONEST READING OF THIS TREND
>
> You cleared an error and lifted a structural sag, and you introduced one warning-level echo doing it. That is a good afternoon — a real net improvement with one small piece of collateral to sweep up. Without CHRONICLE you would have shipped the echo, because it lives three chapters from the win and looks innocent up close. **That** is the value: not the celebration of what cleared, but the catch on what crept in.

## From the dashboard — Ctrl+B Shift+U

You do not have to leave the editor to see any of this. `Ctrl+B Shift+U` opens the CHRONICLE dashboard — the same trend and the same cleared/introduced split,

live, in a panel. It sits under the meta prefix (`Ctrl+B`) because it is a whole-book intelligence, and it reads the very numbers the CLI prints.

● ● ● *The CHRONICLE dashboard — Ctrl+B Shift+U*




The dashboard is where measurement turns back into work. Put the cursor on the introduced echo and press ⏎: the editor jumps straight to the Chapter 4 paragraph carrying it — the same jump the continuity ledger and the read-through give you, so a regression is never more than a keystroke from the prose that holds it. Cut two of the four `fret`s, and the echo is gone. Now the book is genuinely, measurably better than the mark — and you say so with one more keystroke: `m` stamps a fresh milestone on the spot, labelled by today's date.

| | |
|---|---|
| ↑↓ | Scroll the dashboard. |
| ⏎ | Jump to an introduced finding's paragraph — go fix the collateral where it lives. |
| **m** | Mark the current draft now, labelled by today's date. For a chosen name, mark from the CLI instead. |
| **Esc** | Close the dashboard. |

Marking from the dashboard is the **one** place CHRONICLE writes anything interactively, and even then it writes only a milestone — a row of numbers in `chronicle.db`, never a word of your prose. Everything else the panel does is read-only. When you want a milestone under a name of your own rather than a date stamp, mark it from the shell instead — there is no rename; each `mark` records a fresh milestone, so choose the date-stamped `m` or the named CLI form, not both:

● ● ● *inkhaven chronicle mark — naming the revised draft*

```
$ inkhaven chronicle mark "revised-1"
✓ marked "revised-1" — 10 finding(s) (0 error · 4 warn · 6 info)
```

Ten findings now, no errors, and the echo you introduced already swept back up. Two marks in the history, and the afternoon has a settled record.

## Two fixed points — chronicle diff

The bare trend measures **now** against the last mark — it moves as you edit. To compare two **fixed** points in your history, name them both. `chronicle diff <from> <to>` runs the same machinery between two stored milestones rather than against the live book:

● ● ● *inkhaven chronicle diff — first-draft against revised-1*

```
$ inkhaven chronicle diff first-draft revised-1
Chronicle — "first-draft" → "revised-1"

  findings          12 →  10   ▼
  errors             1 →   0   ▼  cleared
  warnings           4 →   4   ·
  infos              7 →   6   ▼

  by category:
    leaked_secret      1 →   0   ▼  cleared
    shape_sag          1 →   0   ▼  cleared

  ✓ 2 cleared    ▲ 0 introduced    · 10 unchanged
```

This is the same revision, told as a settled fact. Notice what changed between the live trend and this diff: `▲ 1 introduced` became `▲ 0 introduced`, and the echo has vanished from the category list entirely. It was born and buried **between** the two marks — introduced by the sag fix, cleared before you stamped `revised-1` — so a diff of the two endpoints never sees it. The live trend caught the echo the instant it existed; the milestone-to-milestone diff is the clean record after you dealt with it. Both are true; they answer different questions. **"What is wrong right now?"** is the trend. **"What did this revision, on balance, do?"** is the diff — two cleared, nothing introduced, the sag and the leaked secret gone for good.

### PURE MEASUREMENT, KEPT APART FROM THE PROSE

Everything in this chapter reads and reports; none of it writes a word of the book. CHRONICLE keeps its milestones in the project's own `chronicle.db`, separate from your manuscript and separate from your `F6` snapshots — two databases, two jobs. It measures whether the revision worked; it never performs the revision. That division is the whole reason you can trust the number: the thing grading the draft has no stake in the grade.

### FICTION

You marked `first-draft` while the mystery still gave itself away and sagged in the middle. The diff proves the reveal now lands where it should and the slack scene tightened — not by your say-so, but by the readers' own count falling. And it caught the echo your fix bred, so the win did not quietly cost you a fault elsewhere.

### NON-FICTION

The same instrument grades an argument. Mark the draft before you restructure a chapter; revise; run `chronicle`. The **cleared** list is the objections you answered and the loose claims you closed; the **introduced** list is the new ambiguity your cut opened two sections down. Proof that you **tightened** the argument rather than merely **churned** it — a number, not a feeling.

The book is better, and now you can say so without flinching. Not because it feels better — because the readers raised eleven findings, then ten, and the two you set out to fix are named in the cleared pile while the one you bred was caught in the introduced pile before it could ship. That is what CHRONICLE is for: it turns **"I**

**think the revision helped"** into **"here is the ledger."** With the draft graded and the collateral swept up, `The Ninth Lantern` is ready to leave the workshop — which is where the last part of this book takes it.

### WHAT YOU LEARNED

- **CHRONICLE** answers the one question no diagnosing reader can: **did the revision work?** It is **pure measurement** — no prose-write path anywhere — kept in the project's own `chronicle.db`, apart from your manuscript and your snapshots.
- A **milestone** freezes the draft's whole verdict: `chronicle mark "first-draft"` runs the shared `collect` once and captures the counts **plus every finding's fingerprint**. Milestones are deliberate — a keystroke, never a daemon; `chronicle list` shows them newest first.
- Bare `inkhaven chronicle` trends the **live** book against your last mark. Every count is **fewer-is-better** — ▼ good, ▲ a regression, · held — and the regressions sort to the top, so `warnings 4 → 5 ▲` never hides behind the wins.
- The signature is the **cleared vs introduced** split — a set difference over stable fingerprints. **Cleared** named the leaked_secret and the shape_sag you resolved (the receipt); **introduced** named the echo a hasty sag-fix bred in ch. 4 (the collateral, itemised and jumpable).
- `Ctrl+B Shift+U` opens the dashboard: ⏎ jumps to an introduced finding to go fix it, `m` marks the draft (date-labelled, renamed via the CLI). It is the one place CHRONICLE writes — and it writes only a milestone, never prose.
- `chronicle diff <from> <to>` compares two **fixed** marks: the settled record. The echo born-and-buried between the marks never appears in it — the live trend catches regressions as they happen, the diff reports the revision on balance.

PART V

# Publishing

CHAPTER 10

# 10

# Assembling the Book

For nine chapters **The Ninth Lantern** has been a tree. You have watched it grow one node at a time: a book, then chapters, then paragraphs, each a small `.typ` file that Inkhaven writes, indexes, and watches while you work. You drafted the cold-lantern opening into it, kept Saltmarch's facts straight against it, planted Aldous's secret so no one acts on it too early, tuned the voices, read the whole thing forward as a first reader, ran the editorial pass, and — last chapter — measured that the revision actually helped. The manuscript is, by any honest reckoning, **done**.

What it is not yet is a **book**. It is still a folder of prose fragments and an index in a database. This chapter is the short walk from one to the other: from the buffer you have been living in to a typeset PDF sitting in the folder you launched from, with

the story's name and the day's date on it. It is the most satisfying two keystrokes in the whole tool, and there is nothing hidden inside them. Every file the process writes is Typst you can open and read; the whole of it is rebuilt from your tree each time you ask, so you never have to wonder whether the PDF matches the manuscript. It always does — it was just made from it.

## The two chords, and the third

Turning a manuscript into a book is a little ladder of three chords, each doing everything the one below it did, and then one thing more. You can reach for them from anywhere — the Tree pane, the Editor, it does not matter — because they act on **the book your cursor is inside**, not on whichever pane happens to have focus.

● ● ● *The three chords, one ladder*

```
Ctrl+B A   Assemble — tree → a Typst-compilable directory.
           No PDF yet, just source a compiler can run.

Ctrl+B B   Build — assemble, THEN compile it to a PDF.
           (A failed compile is handed to the AI.)

Ctrl+B O   Take — build, THEN copy the finished PDF into
           the directory you launched from, timestamped.
```

Read the ladder from the bottom rung up. `Ctrl+B A` is assembly on its own — you reach for it when you want to look at the generated Typst, hand it to `typst watch`, or drive the compiler yourself. `Ctrl+B B` is the everyday chord: assemble and compile in one motion. `Ctrl+B O` — **O** for the copy you **take** away — is the finishing move; it does all of `Ctrl+B B` and then lifts the PDF out of the throwaway build cache and sets it down where your shell can see it.

> **IT HAS TO BE A MANUSCRIPT**
>
> All three chords act on the **user book** your tree cursor is sitting inside. If the cursor is parked on a system book — Notes, Facts, Places, Timeline, and the rest — the chord politely refuses with a status message asking you to pick a real book. Those system books are your author's workshop; they are never part of what ships. (The Timeline is skipped even when it lives inside the manuscript: it is metadata about the story, not the story.)

Whatever you press, Inkhaven saves first. If the paragraph you have open has unsaved edits, they are flushed to disk before assembly reads a single node, so the thing you compile is the thing on your screen. Put your cursor anywhere in **The Ninth Lantern** and press the first chord.

## Assemble — Ctrl+B A

Your manuscript lives as `.typ` files, but not in a shape Typst can compile as it stands. A paragraph file holds only its prose. A chapter is a **folder** and carries no words of its own. Nothing anywhere says "include this one, then that one, then wrap the lot in a book." Assembly's whole job is to synthesise that missing structure — the include tree, the wrappers, the page setup — into a fresh directory, and to leave your actual tree untouched.

It does not write that directory into your project. It writes it to a per-user cache, out of the way, so `git status`, your backups, and shell tab-completion never trip over build litter. When the chord finishes, the status bar prints the absolute path of the one file you would ever name to a compiler — the root `the-ninth-lantern.typ` at the top:

*The assembled tree for The Ninth Lantern*

```
<cache>/inkhaven/artefacts/<project>/the-ninth-lantern/
├── the-ninth-lantern.typ   ← the root; hand THIS to typst
├── globals.typ             ← your wrap_* definitions
├── settings.typ            ← page / fonts / layout
└── book/
    ├── index.typ           ← the book's include list
    ├── 01-the-cold-lantern/
    │   ├── index.typ
    │   ├── 01-a-dark-pillar.typ
    │   └── 02-crane-is-gone.typ
    ├── 02-the-wrong-man/
    │   ├── index.typ
    │   └── 01-toft-and-the-oil.typ
    └── 03-onto-the-mole/
        ├── index.typ
        └── 01-into-the-fret.typ
```

Everything under `book/` mirrors your manuscript one-to-one. Every chapter and subchapter is a folder with its own `index.typ`; every prose paragraph is a `.typ` file; the `NN-` prefixes are the same order numbers the Tree pane shows you. The `index.typ` files are the seams — they `#include` their children and call three helper functions, `wrap_book`, `wrap_chapter`, and `wrap_paragraph`, which are defined once up in `globals.typ`. That is where your typography lives: change what `wrap_chapter` does and every chapter opening in Saltmarch changes with it.

### THE ARTEFACTS TREE IS OUTPUT, NOT SOURCE

Assembly **wipes** `.../the-ninth-lantern/` clean before it writes a byte — so a chapter you deleted, a paragraph you renamed, last week's stale PDF, none of it lingers to confuse the next compile. The cost is that anything you hand-edit in there is gone on the next `Ctrl+B A`. To change your book's look, you edit the `globals.typ` / `settings.typ` paragraphs under the **Typst** system book and re-assemble — never the copies out here. These are disposable.

## Build — Ctrl+B B

`Ctrl+B A` gave you source a compiler can run; `Ctrl+B B` runs it. Press it and a small splash takes over the screen — a spinner and a seconds counter — while Typst lays out the pages. It is quick for a book this size. On success the status bar hands back the path of a real PDF, and **The Ninth Lantern** exists as a document for the first time.

● ● ● *Ctrl+B B — assemble, then compile*

```
        ┌──────────────────────────────────────┐
        │                                      │
        │        ◐  compiling…  1.2 s          │
        │                                      │
        │     the-ninth-lantern · typst        │
        │        Esc to interrupt              │
        │                                      │
        └──────────────────────────────────────┘

  status ▸ PDF ready:
    …/artefacts/…/the-ninth-lantern.pdf
```

Behind that spinner, Inkhaven can drive Typst two ways. By default it finds the `typst` binary on your `PATH` and runs it as a child process — the **external** engine. If you would rather not install a separate tool, an **in-process** engine (`typst_compile.engine = "inprocess"` in your HJSON) links the compiler straight into Inkhaven and needs no external binary at all. The splash names which one is running, so you are never guessing. Assembly, note, never touches Typst; only this compile step does. If the external binary is missing, the compile fails with a clean message pointing you at the install docs — or at the in-process knob.

## When Typst complains

Sooner or later a compile fails, and it is worth seeing that here, because Inkhaven turns the ugliest moment in publishing into something almost pleasant.

Say that between drafts you slipped down into the **Typst** system book and taught your chapter openings to carry an epigraph — a line of the keepers' creed, **no**

**lantern goes dark**, set under each title. You wrote a helper for it in `globals.typ` and called it `wrap_epigraph`. But in the call you added to the cold-lantern chapter you fat-fingered it `wrap_epigrah`. Press `Ctrl+B B` and Typst refuses, because that variable does not exist.

A raw compiler would spit a wall of stderr at you and leave. Inkhaven does two things instead, with no further keystroke. First it scans the error for dangling cross-references — a stray `@fig:` or `@eq:` — and promotes each into a precise finding in the Output pane, because that is a genuine defect, not noise. Then it opens a **fresh** AI chat: it clears the history so nothing dilutes the context, forces the inference mode to Full, and loads a system prompt that knows Typst **and** knows Inkhaven's generated file layout. It packages the stderr with the book's name, slug, and root path, sends it for you, and jumps focus to the AI pane. The answer is already streaming in while you are still reading the error.

● ● ● *A compile failure, handed to the assistant*

```
┌─ AI · llama · streaming · typst-error ──────────────┐
│ you  Book: `The Ninth Lantern` (the-ninth-lantern). │
│      typst compile failed. Smallest concrete fix?   │
│      --- typst stderr ---                           │
│      error: unknown variable: wrap_epigrah          │
│        ┌─ book/01-the-cold-lantern/index.typ:7:4    │
│                                                     │
│ ai   Line 7 calls `wrap_epigrah` — a typo for       │
│      `wrap_epigraph`. But you never edit these      │
│      generated files; they are rebuilt every run.   │
│      Fix the call in the chapter paragraph under    │
│      the Typst system book (the epigraph you added  │
│      to "The Cold Lantern"), then re-assemble.      │
└─────────────────────────────────────────────────────┘
```

That last sentence is the whole point. The error names a file down in the artefacts tree, but that file is generated — editing it would be undone on the next assembly. The assistant knows this and walks you back to the real source: the paragraph in the Typst book where you wrote the call. And because a failed compile **keeps** the assembled tree rather than deleting it, you can also open that `index.typ` at the very line Typst named, read it in context, and confirm the diagnosis with your own eyes. Fix the source paragraph, press `Ctrl+B B` again, and the book compiles.

**FICTION**

You added an epigraph helper and mis-typed its name; the fix lives in your Typst book, not in the failing generated file. Same for a missing figure or a broken `wrap_chapter` — chase it back to the paragraph you actually wrote.

**NON-FICTION**

A non-fiction build fails the same friendly way — a malformed `@key` citation, a table macro called wrong — and the same fresh, layout-aware chat meets you with the smallest fix. Nothing about this path is fiction-only.

## Take the book — Ctrl+B O

The PDF now exists, but it exists in a cache directory with a `<hash>` in the path that no one wants to type. `Ctrl+B O` is the last rung: it does everything `Ctrl+B B` does, and then **copies** the finished PDF out of the cache and into the directory you launched Inkhaven from — under a name you can actually find:

*What Take drops in your launch directory*

```
the-ninth-lantern-20260508-1730.pdf
```

The stem is the book's slug, `the-ninth-lantern`; the stamp is `YYYYDDMM-HHMM` — year, day, month, hour, minute, the same ordering Inkhaven uses for its backup files. Take **copies**, it does not move: the original stays in the artefacts cache too, so a later imposition pass or a re-take still has a source PDF to work from. The status bar reports both the delivered path and the source, and there is your book, sitting in the folder where you started the morning:

● ● ● *Back in the shell where it all began*

```
~/writing/ninth-lantern $ ls
inkhaven.hjson   books/   .inkhaven/

~/writing/ninth-lantern $ inkhaven          # ... write the book ...
                                            # ... Ctrl+B O ...
~/writing/ninth-lantern $ ls *.pdf
the-ninth-lantern-20260508-1730.pdf

~/writing/ninth-lantern $ open the-ninth-lantern-20260508-1730.pdf
```

That is the buffer-to-book moment: you launched Inkhaven in an empty folder ten chapters ago, and now a timestamped PDF of **The Ninth Lantern** is sitting right next to the project you built it from. If you have configured extra formats (Markdown, TeX, EPUB — the next chapter's business), Take writes them beside the PDF with the same stem; an extra that fails is reported but never aborts the take, because the PDF you actually asked for is already on disk before the extras run.

## The same path, from the command line

Everything the three chords do is available without the TUI at all — for a build script, a CI job, or just checking that a typography change still compiles. `inkhaven build` is the headless mirror of the ladder:

● ● ● *Building The Ninth Lantern from the shell*

```
$ inkhaven build
Assembling `The Ninth Lantern` (slug: the-ninth-lantern)…
  [12/28] book/01-the-cold-lantern/01-a-dark-pillar.typ
Assembly OK · root: …/the-ninth-lantern.typ (28 files)

$ inkhaven build --compile
PDF: …/the-ninth-lantern.pdf
```

Without `--compile` it stops after writing the artefacts tree — that is `Ctrl+B A`. With it, it runs `typst compile` and prints **only** the final PDF path to stdout; the per-file progress goes to stderr, so the command drops cleanly into a pipeline. The `--book-name` flag is optional when the project has exactly one user book, as ours

does, and required when it has more. This is the one place a build can run untended — which is exactly why it is worth having when the manuscript is stable and the only question left is **did the last change still build?**

### WHAT YOU LEARNED

- Three chords climb one ladder: **`Ctrl+B A`** assembles the tree into a Typst-compilable directory, **`Ctrl+B B`** also compiles it to a PDF, and **`Ctrl+B O`** also copies that PDF — timestamped `the-ninth-lantern-YYYYDDMM-HHMM.pdf` — into the folder you launched from.
- Assembly writes a **fresh, disposable** tree under a per-user cache: a root `the-ninth-lantern.typ`, `globals.typ`, `settings.typ`, and a `book/` subtree of `index.typ` files calling `wrap_book` / `wrap_chapter` / `wrap_paragraph`. It wipes that directory every run — treat it as output, never source.
- Change your book's look in the **Typst system book** paragraphs (`globals.typ` / `settings.typ`) and re-assemble; never edit the artefacts copies, which are overwritten each build.
- A failed compile **routes its stderr into a fresh, Typst-aware AI chat** automatically — cleared history, Full mode — and walks you back to the real source paragraph; the assembled tree is kept so you can read the offending line in context.
- `inkhaven build [--compile]` is the headless mirror of the chords, for CI and scripts: progress to stderr, the final PDF path to stdout.

CHAPTER 11

# 11

# Out Into the World

**The Ninth Lantern** has a PDF. The last chapter walked the tree into a Typst directory and let `typst compile` turn it into a typeset book — the destination prose has had since the first printing press. It is the format you send to a printer, and the one you are proud to hold.

But a printed page is only one of the places a reader lives. One reader opens a book on a phone on the train; another opens it in a browser and never downloads anything at all. Neither wants a fixed A5 page zoomed to a thumbnail. They want prose that **reflows** to the glass in front of them. So before we call the book finished, we send it to the two other formats a reader actually uses — an EPUB for the e-

reader, and a small web edition for the browser — and we do it, as Inkhaven does everything at the end, with a single command each.

## An EPUB the e-reader opens

An EPUB is what an e-reader consumes: underneath, a zip of web pages with a **spine** that orders them and a navigation document that indexes them. Inkhaven writes one directly — no `pandoc`, no external converter — because it already holds your prose as Typst and carries a `zip` library in the binary. For a project with a single user book, the command needs nothing but its name:

● ● ● *Exporting the finished e-book*

```
$ inkhaven epub
  embedding cover (image/jpeg, 96 KB)
Exporting `The Ninth Lantern` → EPUB (7 chapters)…
EPUB: the-ninth-lantern.epub (7 chapters · 148 KB)
```

Three lines, and there is the whole shape of the export in them. Read them in order.

**It found the cover.** Before it wrote a single chapter, `inkhaven epub` looked in the project root, found the `cover.jpg` we dropped there, and reported the format and size as it embedded it. That is the entire ritual: drop a `cover.jpg` or `cover.png` beside `inkhaven.hjson`, and it becomes both the thumbnail a reader's library shows and the opening page of the book. No flag, no config.

**One spine document per chapter.** The middle line is the important one. A real e-book is not one long scroll — it is a chapter the reader can jump to from the table of contents, a place their progress bar can measure against. So `inkhaven epub` writes **one spine document per Chapter node**, in display order: `The Cold Lantern` becomes `chapter-001.xhtml`, the next becomes `chapter-002.xhtml`, and so on down our seven. Each holds that chapter's paragraphs converted to XHTML, with any subchapter titles surfaced as headings inside the flow. The reader gets a proper spine; the e-reader gets its jump points.

**The result reflows.** The last line is the payoff. Unlike the PDF, nothing in the EPUB is pinned to a page. The e-reader lays the words out to its own screen at its own type size — that is what "reflowable" means, and it is the whole reason the

format exists. The 148 KB is small because there are no fixed pages to store, only prose and the structure that orders it.

### What rode along inside

The archive `inkhaven epub` produced has exactly the shape the EPUB spec demands, and you never have to think about any of it: the `mimetype` entry first and uncompressed, a `META-INF/container.xml`, a package document listing every chapter in its manifest and spine, an EPUB 3 `nav.xhtml` for modern readers and an EPUB 2 `toc.ncx` for older ones, a stylesheet, the cover, and the seven chapter files.

● ● ● *Inside the-ninth-lantern.epub*

```
mimetype                 (stored, first — the EPUB rule)
META-INF/container.xml
OEBPS/content.opf        (metadata + manifest + spine)
OEBPS/nav.xhtml          (EPUB 3 navigation)
OEBPS/toc.ncx            (EPUB 2 back-compat)
OEBPS/style.css
OEBPS/cover.jpg
OEBPS/cover.xhtml
OEBPS/chapter-001.xhtml
OEBPS/chapter-002.xhtml
...
```

The converter is honest about the markup our prose actually used. The `= title` line each paragraph carries — the scene label you saw in the Tree, "The Cold Lantern" — is **scaffolding**, so it is stripped, and the reader sees flowing chapter prose rather than "001." headings. `_emphasis_` becomes an italic, `*strong*` a bold, blank-line-separated blocks become paragraphs, and the XML special characters are escaped. And the one footnote in Aldous's chapter became a proper EPUB 3 **popup**: a numbered reference in the flow and a collected note at the chapter's end, which Apple Books renders as a tap-to-pop and plainer readers show at the foot. The metadata — title, author (from your `editor.comment_author` config), language, a stable identifier — was filled in for you. The title and author each take a flag if you want to override them (`--title`, `--author`); the language follows your project `language` and the identifier is generated for you.

> **TWO EPUB PATHS, ONE TO PREFER**
>
> There is also a minimal `inkhaven export epub`, which packs the whole book as a single-chapter EPUB so it matches the Markdown dump byte for byte — handy in a batch build, but not what a reader wants. For the real e-book — chapters as separate spine documents, the cover, popup footnotes — use the dedicated `inkhaven epub`, the command we just ran. When in doubt, that is the one.

## A small web edition

The second reader never downloads a file at all — she follows a link. For her we build a website straight from the same book. One command, one directory:

*Building the web edition*

```
$ inkhaven export html -o site/
wrote HTML site to site/

$ ls site/
index.html                    ch04-the-reveal.html
ch01-the-cold-lantern.html    search.js
ch02-a-suppliers-fault.html   search-index.js
ch03-onto-the-mole.html       theme.css
...
```

What lands in `site/` is a **self-contained static website** — one HTML page per chapter (`ch01-the-cold-lantern.html`, named from the chapter's title), an `index.html` built from the book's front matter, a navigation list linking them all, and a `theme.css` for the look. It is the third consumer of the same node tree the PDF and the EPUB were built from, localised to the book's language, with any images copied in alongside. You can open `index.html` from disk, or drop the whole folder on any host that serves files; there is nothing to install and nothing to run.

The two files worth pointing at are `search.js` and `search-index.js`. Together they give the site a **search box that works with no server behind it**. Inkhaven writes the book's text into `search-index.js` as a plain script the page

loads directly — which is why the search runs even from a `file://` URL, opened off a USB stick with no network in sight — and `search.js` is the small handful of vanilla JavaScript that matches what you type against it, titles first. No library, no query going out to anyone. A reader can stand in the middle of **The Ninth Lantern** and jump to every mention of the sea-fret in the time it takes to type the word.

> **MAKE THE SITE YOUR OWN**
>
> The look is driven by **overridable templates**: point `--templates <dir>` at your own files (or scaffold the defaults with `--eject-templates <dir>` and edit those), and anything you leave out falls back to the bundled default, so a theme can be one stylesheet. A whole companion — **Publish Your Book to the Web** — covers styling, templates, appendix pages, and going live. This section is the map; that book is the territory.

> **FICTION**
>
> Same two commands, no change. **The Ninth Lantern** ships as an EPUB a reader buys and opens on a phone, and as a small site you point people to — the cover, the seven chapters, the searchable prose, all from the one tree you wrote.

> **NON-FICTION**
>
> Identical path, different shelf. A non-fiction book becomes the same EPUB for a reader's library; and `export html` is how a manual or a set of docs becomes a **documentation site** — chapters as pages, the client-side search doing the work an FAQ never could.

## From a blank project to a finished book

Step back and look at the whole distance. Eleven chapters ago there was nothing — an empty project on a Monday morning, one `inkhaven init` and a blinking Tree. There is now a finished book in the three formats a book actually reaches a reader in: a typeset **PDF** for the page and the printer, an **EPUB** for the e-reader, and a **web edition** for the browser. One manuscript, one tree, three destinations, each a single command at the end.

*The distance we came*

```
I    Foundation .. init · the world & the cast
II   Drafting .... the opening · the facts kept straight
III  The Middle .. the secret (KEN) · voices & threads
IV   Revision .... read-through · editorial pass ·
                   did it get better?
V    Publishing .. the PDF · the EPUB · the web
```

And every feature we passed had its place in that arc. We reached for the world and the cast when the story needed somewhere to stand, for the who-knows-what reader when a secret went in that no one could act on early, for the character voices when five people had to sound like five people, for the read-through and the editorial pass when the draft was whole enough to judge, and for "did it get better?" to prove the revision had earned its keep. Not one of them was reached for its own sake. That is the thing this book set out to show: Inkhaven is not a pile of features to work through, but a set of tools that each answer a question a real manuscript asks — and it asks them in roughly this order, every time.

You will not use all of them on **your** book; few books need all of them. But you have now seen where each one lives in the life of a manuscript, and that is enough to reach for the right one at the right moment — and to know which ones to leave on the shelf. For anything we touched only in passing — the DOCX an agent expects, the arXiv bundle a preprint server compiles, the audiobook a reader listens to on a walk — **The Inkhaven Manual** is the reference waiting on the next shelf, one chapter per feature, ready when your book asks a question this one did not.

That is the journey. A blank project became a book, and the book has gone out into the world. Go and write yours — Inkhaven will be there for each question it raises, in about the order you have just seen. The lantern is lit; the fog is someone else's problem now.

### WHAT YOU LEARNED

- `inkhaven epub` writes a real EPUB 3 for the e-reader — **one spine document per chapter**, a zero-config `cover.jpg`, popup footnotes, and prose that **reflows** to the reader's screen — from a single command, no external tools.
- `inkhaven export html -o <dir>` builds a **self-contained static website** — one page per chapter, an index, and a **client-side search** (`search.js` + `search-index.js`) that runs with no server, even from a `file://` URL; `--templates` / `--eject-templates` make the look yours.
- The same two commands serve non-fiction just as well — the EPUB for a reader's library, `export html` for a documentation site.
- We began at a blank project and ended with one book in **three** formats — PDF, EPUB, and web — and every feature along the way had its place in that arc.
- For anything touched only in passing — DOCX, the arXiv bundle, the audiobook — **The Inkhaven Manual** is the per-feature reference. Now go write yours.

AFTERWORD

# About the author

*Vladimir Ulogov.*

Vladimir Ulogov has spent decades building infrastructure for distributed systems — the kind of software that watches other software. Early in his career he worked on monitoring and telemetry platforms; later years took him into federated observability, telemetry buses, and the architecture of systems that have to make sense of millions of data points without losing the thread.

Observability, in the end, is the art of **holding a system too large for one head** — of watching it without losing the thread, and never reporting a thing it cannot back up. It is not an accident that a tool he built for writers carries the same instinct to the desk: a book, past a certain size, is exactly such a system, and it too

deserves an instrument that holds it for you.

What makes him slightly unusual in his corner of the industry is a tendency to write his own tools — not in the sense of small utilities, but in the sense of programming languages. The Bund language (its compiler, its VM, its document store, its parser) lives in a long series of Rust crates on crates.io. `rust_dynamic`, `rust_multistack`, `rust_multistackvm`, `bundcore`, `zbus_universal_data_gateway` — each is a building block that exists because the off-the-shelf options didn't fit the shape of the work.

## Why Inkhaven exists

Inkhaven is Vladimir's personal reflection on how a single tool can hold the **whole** of writing a book. The frustration it answers is one every long-form writer knows: that the work is scattered across a dozen apps that don't talk to each other — a browser for research, one program for the prose, another for the characters, a spreadsheet for the word count, a design tool for the final typesetting — and every seam between them is a place the book leaks.

Inkhaven's wager is that all of it belongs in one place, next to the keyboard: the prose and the world it is set in, the facts it must not contradict, the assistant you steer, and the finished PDF. Not because integration is elegant, but because a book past a certain size outgrows the mind writing it, and a tool that holds your words should also be able to **hold the book** — its facts, its continuity, its shape — so the writer is free to do the one thing no tool can, which is write.

## A work of love

Inkhaven is open source. The licence is permissive — you can read it, fork it, study it, modify it, and pass it on. Strictly speaking, the licence also lets you sell it. The author would, gently but firmly, disagree with you doing that. Inkhaven was not designed as a *for sale* project. It is a work of love made for the authors who can least afford to pay for software, and turning it into a commercial product would betray the reason it exists. So please — don't.

It carries no analytics, no telemetry, no upsell. The binary will never phone home; the project will never have an "Enterprise" tier you have to escape from.

This was a deliberate choice. There are excellent commercial tools for writing. They cost money — which is fine for many writers, and a barrier for many more. Inkhaven exists for the second group: for the graduate student writing a dissertation on a battered laptop, for the novelist who shouldn't have to pick between rent and software, for the engineer drafting in the same terminal where they already write code, for anyone who would benefit from a tool that respects their work without asking for a credit-card number.

## A note on cooperation

Vladimir believes firmly in the human capacity for mutual help — that we make better work, and live better lives, when we share what we know and what we build. Open source is one of the most concrete expressions of cooperation our era has produced: code read, improved, and passed forward without payment, without permission, by people who will never meet.

If Inkhaven helps you finish your book — that is enough. If it gives you a chord pattern you adapt into your own tool — that's a gift back to the larger project of making software writers can love. As a society, we achieve the greatest things when we help each other rather than compete with each other.

## Where to find more

| | |
|---|---|
| **GitHub** | @vulogov — the source for Inkhaven, Bund, and the dozen-plus Rust crates that carry the infrastructure. Issues and PRs welcome. |
| **LinkedIn** | /in/vladimirulogov — posts on observability, the occasional long-form essay. |
| **YouTube** | @vulogov — talks and walkthroughs from the conference trail. |
| **X / Twitter** | @vladimir_ulogov |

*If a fact you grounded ever surprises you, or a command trips you up and you can't find the answer in this book — open an issue on GitHub. The author reads them.*

END OF THE BOOK · INKHAVEN 3.0.0